Quantum Error Correction

Lecture Notes — Version 3.0

Extended Edition with Worked Examples and Visual Aids

Quantum Computing Course

June 1, 2026

Abstract

These lecture notes provide a comprehensive, self-contained introduction to quantum error correction (QEC) aimed at undergraduate students with basic linear algebra and introductory quantum mechanics. We begin with the question of *why* quantum information is fragile, build from classical intuition to the 3-qubit bit-flip and phase-flip codes, study the Shor 9-qubit code and the Steane $[\![7, 1, 3]\!]$ code, develop the stabilizer formalism as a unifying language, and then move to the topological surface code. Fault-tolerant gates, magic state distillation, the threshold theorem, and advanced quantum LDPC and color codes round out the material.

Contents

1 Introduction: Why Quantum Error Correction?

Intuition

Imagine writing a message on a whiteboard in a room with a randomly blowing fan. Every few seconds a gust might smudge a letter. After 1000 letters the message is unreadable. Classical computers deal with this by building transistors so reliable that the "fan" almost never blows. Quantum computers *cannot* do the same thing: the "fan" is the quantum environment itself, and it can never be fully shut out. Quantum error correction is the strategy of encoding the message so cleverly that even with constant smudging, the original meaning can always be recovered.

Quantum systems are notoriously fragile. Qubits, unlike classical bits, are susceptible to **decoherence** and operational errors from two main sources:

- **Decoherence.** A qubit interacts with its surroundings (stray magnetic fields, vibrations, nearby electrons). This interaction *entangles* the qubit with the environment, causing the delicate quantum superposition to leak away. The characteristic times are called T_1 (energy relaxation) and T_2 (dephasing). For state-of-the-art superconducting qubits, $T_2 \sim 100\,\mu$s; a typical gate takes ~ 50 ns, so only about 2000 gates are possible before significant decoherence.
- **Gate errors.** No physical operation is perfect. A CNOT gate might have a 0.1–1% error probability. For a 1000-gate algorithm on a single unprotected qubit with gate error $p = 0.001$, the probability of *no error at all* is $(1-0.001)^{1000} \approx 0.37$. The chance of getting the correct answer with high probability is essentially zero.

Key Point

The goal of quantum error correction is to **detect and correct errors without measuring (and thus collapsing) the quantum information**. This is fundamentally different from classical error correction, and requires genuinely new ideas.

1.1 A Concrete Failure Rate Calculation

Let us make the problem concrete before developing solutions.

Worked Example 1.1: Failure probability accumulation

Suppose we run an algorithm that requires $n = 1000$ sequential single-qubit gates. Each gate has error probability $p = 0.01$ (1%). What is the probability that at least one gate fails?

Solution. The probability that a *single* gate succeeds is $1 - p = 0.99$. The probability that *all* n gates succeed is:

$$P(\text{no error}) = (1-p)^n = (0.99)^{1000} \approx e^{-10} \approx 4.5 \times 10^{-5}.$$

So with overwhelming probability ($\approx 99.995\%$), at least one gate fails. The computation is useless without error correction.

For comparison, if we could reduce the effective logical error rate to $p_L = 10^{-6}$ using QEC (a reasonable target for near-future devices), the same 1000-gate algorithm succeeds with probability $(1 - 10^{-6})^{1000} \approx 0.999$, which is excellent.

1.2 Why Classical Error Correction is Not Enough

Classical error correction often uses **repetition codes** or parity checks. For example, a classical 3-bit repetition code encodes:

$$0 \longrightarrow 000, \qquad 1 \longrightarrow 111,$$

and recovers from a single bit-flip using majority voting. However, applying this idea directly to qubits fails for three fundamental reasons:

1. **The no-cloning theorem** (§1.3) forbids copying an unknown quantum state. We cannot produce three copies of $|\psi\rangle$ to store redundantly.
2. **Errors are continuous.** A qubit can be "partially rotated" by any angle θ. There is an uncountably infinite number of possible errors, unlike the two classical errors (0 and 1).
3. **Measurement destroys superpositions.** If we try to "look at" a qubit to check for errors, we collapse its state.

1.3 The No-Cloning Theorem

Theorem 1.1 (No-Cloning Theorem)**.** *There is no unitary operation U such that for* all *quantum states $|\psi\rangle$:*

$$U\big(|\psi\rangle \otimes |0\rangle\big) = |\psi\rangle \otimes |\psi\rangle\,.$$

Proof by contradiction. Suppose such a U exists. Let $|\psi\rangle$ and $|\phi\rangle$ be any two single-qubit states. By assumption:

$$U(|\psi\rangle\,|0\rangle) = |\psi\rangle\,|\psi\rangle\,, \qquad U(|\phi\rangle\,|0\rangle) = |\phi\rangle\,|\phi\rangle\,.$$

Since U is unitary it preserves inner products:

$$\langle\psi|\,\langle 0|0\rangle\,|\phi\rangle = \big(\langle\psi| \otimes \langle\psi|\big)\big(|\phi\rangle \otimes |\phi\rangle\big),$$

which simplifies to

$$\langle\psi|\phi\rangle = \langle\psi|\phi\rangle^2\,.$$

This forces $\langle\psi|\phi\rangle \in \{0, 1\}$. Since $|\psi\rangle$ and $|\phi\rangle$ were *arbitrary*, this is a contradiction: two distinct non-orthogonal states (e.g. $|0\rangle$ and $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$) have $\langle 0|+\rangle = \frac{1}{\sqrt{2}} \neq 0, 1$. □

Intuition

The no-cloning theorem does *not* say we cannot protect quantum information — it says we cannot protect it by simply making copies. Quantum error correction instead spreads the information across an *entangled* state of multiple qubits. No single qubit holds a copy; the information is holographically encoded in the correlations.

1.4 The Three-Step Strategy of QEC

Every quantum error correcting code works through three steps:

1. **Encode.** Map one logical qubit $\alpha\,|0\rangle + \beta\,|1\rangle$ into an entangled state of n physical qubits. The coefficients α, β are *not* stored in any single qubit; they live in global correlations.
2. **Syndrome measurement.** Use ancilla (helper) qubits to measure certain *parities* of the data qubits. These parity measurements (called **syndromes**) reveal *what* error occurred without revealing *what* α and β are.

3. **Recovery.** Apply a corrective operation based on the syndrome. Because the syndrome uniquely identifies the error (for correctable errors), we can undo the damage and restore the original encoded state exactly.

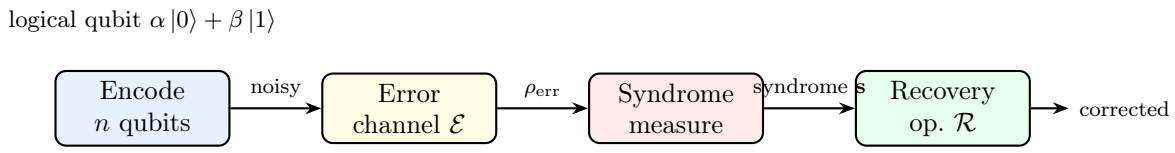


Figure 1: The three-step QEC pipeline. The logical qubit is first encoded into n physical qubits. After an error channel acts, the syndrome is measured (without disturbing α, β), and a recovery operation restores the original state.

1.5 Historical Context

Quantum error correction emerged in the mid-1990s in rapid succession:

- **Shor (1995):** First QEC code — a 9-qubit code correcting arbitrary single-qubit errors.
- **Steane (1996):** 7-qubit CSS code derived from classical Hamming codes; simpler and more efficient.
- **Knill & Laflamme (1997):** General algebraic conditions (the KL conditions) characterising when a set of errors is correctable.
- **Kitaev (1997):** Toric/surface code — topological QEC with local stabilizers, high threshold, and deep connections to topology.
- **Preskill (1997–2012):** Threshold theorem and fault-tolerant computing framework.

1.6 Learning Goals for This Chapter

After reading this chapter, you should be able to:

- Calculate the probability that an unprotected quantum computation fails due to accumulated errors.
- State and prove the no-cloning theorem and explain why it does *not* forbid QEC.
- Describe the encode–syndrome–recover strategy in plain language.
- Explain the three fundamental obstacles preventing direct application of classical error correction to qubits.

Exercise 1.1. A quantum algorithm requires $n = 500$ two-qubit gates, each with error probability $p = 0.005$. What is the probability that no error occurs? What is the minimum gate fidelity required to ensure the computation succeeds with probability at least 0.9?

Exercise 1.2. Prove that the no-cloning theorem forbids *broadcasting*: there is no quantum channel $\mathcal{E}$ such that $\mathcal{E}(|\psi\rangle\langle\psi|) = |\psi\rangle\langle\psi| \otimes |\psi\rangle\langle\psi|$ for all pure states $|\psi\rangle$. (*Hint:* use a linearity argument rather than unitarity.)

Exercise 1.3. Explain in your own words why the no-cloning theorem does *not* prevent quantum error correction. What feature of QEC allows it to protect information without copying it?

Section Summary

Section 1 key points:

- Qubit errors accumulate rapidly; even 0.1% gate error makes a 1000-gate algorithm unreliable without correction.
- No-cloning theorem: we cannot copy an unknown quantum state.
- QEC works by encoding into entangled states, measuring syndromes (error indicators without revealing logical information), and applying corrective operations.

2 Quantum Errors and the Pauli Framework

Intuition

A single qubit lives on the surface of the Bloch sphere: the north pole is $|0\rangle$, the south pole is $|1\rangle$, and every other point is a superposition. An error is a rotation of this sphere. Every rotation decomposes into rotations about three coordinate axes, generated by X (which swaps $|0\rangle \leftrightarrow |1\rangle$), Z (which flips the relative phase), and Y (which combines both effects). This decomposition turns *infinitely many possible errors* into a *finite correctable list* — but to understand *why* it works, we need to build up the picture carefully from physical first principles.

2.1 Single-Qubit Error Types

Consider a qubit in state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$. Although a qubit can undergo infinitely many possible errors (arbitrary rotations or phase changes), it is sufficient to study a small set of fundamental error operators. The Pauli operators $\{I, X, Y, Z\}$ form a **basis** for all 2×2 complex matrices — this means any physical error operator can be expressed as a linear combination of them.

We therefore focus on the following three "pure" error types:

Error	**Op.**	**Action**	**Physical meaning**
Bit-flip	X	$\|0\rangle \leftrightarrow \|1\rangle$	Like a classical bit-flip; exchanges amplitudes
Phase-flip	Z	$\|0\rangle \mapsto \|0\rangle$, $\|1\rangle \mapsto -\|1\rangle$	Flips the relative phase between basis states
Bit+Phase	$Y{=}iXZ$	$\|0\rangle \mapsto i\|1\rangle$, $\|1\rangle \mapsto -i\|0\rangle$	Combines both bit-flip and phase-flip effects

2.2 Pauli Matrices

The four **Pauli operators** are:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

They are Hermitian ($P^\dagger = P$), unitary ($P^\dagger P = I$), and traceless for $P \in \{X, Y, Z\}$. Their key algebraic properties are:

$$X^2 = Y^2 = Z^2 = I, \tag{1}$$

$$XY = iZ, \quad YZ = iX, \quad ZX = iY, \tag{2}$$

$$\{X, Y\} = \{Y, Z\} = \{X, Z\} = 0 \quad \text{(every pair of distinct Paulis anticommutes).} \tag{3}$$

Worked Example 2.1: Verifying Pauli algebra

Let us verify $ZX = iY$ and $XZ = -iY$ directly:

$$ZX = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \qquad iY = i\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix},$$

so $ZX = iY$. Then:

$$XZ = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = -ZX = -iY,$$

confirming $\{X, Z\} = XZ + ZX = -iY + iY = 0$. □

2.3 Discretisation of Quantum Errors

Theorem 2.1 (Discretisation of Errors)**.** *Any single-qubit error E can be written as a linear combination of Pauli operators:*

$$E = c_0\, I + c_1\, X + c_2\, Y + c_3\, Z, \qquad c_0, c_1, c_2, c_3 \in \mathbb{C}.$$

*Consequently, if a quantum code can correct all single-qubit Pauli errors $\{I, X, Y, Z\}$, it can correct **any** single-qubit error.*

Proof sketch. The four Pauli matrices are linearly independent and the space of 2×2 complex matrices has dimension 4, so $\{I, X, Y, Z\}$ is a basis for that space. Any 2×2 matrix E can therefore be decomposed uniquely in this basis. When the syndrome is measured, the measurement projects the error onto one of its Pauli components; that component is a pure Pauli error which the code corrects. □

Intuition

Your code knows how to handle three "standard" faults (X, Y, Z). When some exotic fault E happens, *the act of checking which standard fault occurred* forces the actual error to collapse into one of those standard faults. After the check, you have an ordinary fault you already know how to fix. This is the magic of quantum measurement used constructively.

Worked Example 2.2: Decomposing a rotation error

A common physical error is an **over-rotation**: the qubit rotates by θ about the X-axis. The error operator is:

$$E(\theta) = e^{-i\theta X/2} = \cos\tfrac{\theta}{2}\, I \;-\; i \sin\tfrac{\theta}{2}\, X.$$

The coefficients are $c_0 = \cos(\theta/2) \in \mathbb{R}$ and $c_1 = -i\sin(\theta/2) \in i\mathbb{R}$; note that c_1 is purely imaginary, illustrating that the coefficients c_i are genuinely complex. For $\theta = 0.1\,\text{rad}$: $c_0 \approx 0.9988$ and $c_1 \approx -0.0500i$. When the syndrome is measured, the error collapses to I with probability $|c_0|^2 \approx 0.9975$ or to X with probability $|c_1|^2 \approx 0.0025$, and the code handles both outcomes correctly.

2.4 Quantum Channels and Kraus Operators

2.4.1 Where do Kraus operators come from?

So far we have described errors as single Pauli operators acting on a qubit. But in reality, a qubit does not sit in isolation — it continuously interacts with its surrounding environment (stray electric fields, phonons, neighbouring electrons). We need a formalism that captures this messy, probabilistic, many-body physics in a compact mathematical form. That formalism is the **quantum channel**, described by Kraus operators.

Intuition

Imagine placing your qubit in a box with a tiny hole. Unknown signals leak in and out, disturbing the qubit in unpredictable ways. You cannot track what goes through the hole. A quantum channel describes exactly how the qubit's state changes when you average over all the disturbances you *cannot* see.

2.4.2 Derivation of the Kraus Representation

Suppose the qubit starts in state ρ. We use the **density matrix** ρ (rather than a state vector $|\psi\rangle$) because after tracing out the environment the qubit will generally be in a *mixed* state — a classical probability distribution over quantum states — which density matrices describe and pure state vectors cannot. For a pure initial state, $\rho = |\psi\rangle\langle\psi|$. The environment starts in a known pure state $|e_0\rangle$. Together, the combined system evolves unitarily under U:

$$\rho \otimes |e_0\rangle\langle e_0| \xrightarrow{U} U\big(\rho \otimes |e_0\rangle\langle e_0|\big)U^\dagger.$$

We do not observe the environment, so we *trace it out* by summing over all environmental basis states $\{|e_k\rangle\}$:

$$\mathcal{E}(\rho) = \mathrm{Tr}_{\mathrm{env}}\Big[U\big(\rho \otimes |e_0\rangle\langle e_0|\big)U^\dagger\Big] = \sum_k \langle e_k|\, U\, (\rho \otimes |e_0\rangle\langle e_0|)\, U^\dagger\, |e_k\rangle\,.$$

Define the **Kraus operator** for the k-th environmental outcome:

$$\boxed{K_k \;\equiv\; \langle e_k|\, U\, |e_0\rangle\,.}$$

This is a 2×2 matrix acting on the qubit alone: U acts on the joint (qubit $\otimes$ environment) space; contracting the environment indices with $\langle e_k|$ and $|e_0\rangle$ traces away the environment and leaves a qubit operator. Substituting this definition:

$$\boxed{\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger.}$$

Why the completeness condition $\sum_k K_k^\dagger K_k = I$? Unitarity of U gives $U^\dagger U = I_{\mathrm{total}}$. Using the completeness relation $\sum_k |e_k\rangle\langle e_k| = I_{\mathrm{env}}$:

$$\sum_k K_k^\dagger K_k = \sum_k \langle e_0|\, U^\dagger\, |e_k\rangle\langle e_k|\, U\, |e_0\rangle = \langle e_0|\, U^\dagger \left(\sum_k |e_k\rangle\langle e_k|\right) U\, |e_0\rangle = \langle e_0|\, U^\dagger U\, |e_0\rangle = I.$$

This ensures $\mathrm{Tr}[\mathcal{E}(\rho)] = \mathrm{Tr}[\rho] = 1$: probabilities sum to 1.

2.4.3 Physical Meaning of Each Kraus Operator

Each K_k describes what happens to the qubit *given* that the environment jumped to state $|e_k\rangle$:

- The probability of this jump is $p_k = \mathrm{Tr}[K_k \rho K_k^\dagger]$.
- The normalised post-error qubit state, if we knew this jump occurred, would be $K_k \rho K_k^\dagger / p_k$.
- Since we *do not* know which jump occurred, we add all outcomes incoherently: $\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger$.

"Average over all the ways the environment could have disturbed the qubit, weighted by how likely each disturbance is."

Caution

The Kraus decomposition is **not unique**. Sets $\{K_k\}$ and $\{\tilde{K}_k\}$ related by $\tilde{K}_k = \sum_j u_{kj} K_j$ (with u unitary) describe the *same* channel, corresponding to different bases for the environment's Hilbert space. The channel $\mathcal{E}$ itself is unique.

2.4.4 Key Examples

Bit-flip channel (probability p).

$$K_0 = \sqrt{1-p}\, I, \quad K_1 = \sqrt{p}\, X. \qquad \mathcal{E}(\rho) = (1-p)\rho + p\, X\rho X.$$

Completeness: $(1-p)I + pI = I$. ✓

Dephasing channel.

$$K_0 = \sqrt{1-p}\, I, \quad K_1 = \sqrt{p}\, Z. \qquad \mathcal{E}_{\text{deph}}(\rho) = (1-p)\rho + p\, Z\rho Z.$$

Dephasing contracts the equatorial plane of the Bloch sphere: the x- and y-components of the Bloch vector shrink while the z-component is unchanged.

Depolarising channel.

$$\mathcal{E}_{\text{dep}}(\rho) = (1-p)\rho + \tfrac{p}{3}(X\rho X + Y\rho Y + Z\rho Z),$$

$$K_0 = \sqrt{1-p}\, I, \quad K_1 = \sqrt{p/3}\, X, \quad K_2 = \sqrt{p/3}\, Y, \quad K_3 = \sqrt{p/3}\, Z.$$

Amplitude damping channel. This models energy relaxation (T_1 decay): the qubit decays from $|1\rangle$ to $|0\rangle$ with probability $\gamma = 1 - e^{-t/T_1}$:

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \qquad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix}.$$

- K_0 (no-jump): leaves $|0\rangle$ unchanged; contracts $|1\rangle$ by $\sqrt{1-\gamma}$, reflecting reduced probability of remaining excited.
- K_1 (jump): maps $|1\rangle \mapsto \sqrt{\gamma}\,|0\rangle$ and annihilates $|0\rangle$; this is the discrete event of photon emission with probability γ.

Completeness: $\begin{pmatrix} 1 & 0 \\ 0 & 1-\gamma \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & \gamma \end{pmatrix} = I$. ✓

Worked Example 2.3: Depolarising channel on $|+\rangle$

Let $\rho = |+\rangle\langle+| = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ and $p = 0.12$, so $1-p = 0.88$ and $p/3 = 0.04$:

$$X\rho X = \rho \quad (|+\rangle \text{ is a } +1 \text{ eigenstate of } X),$$

$$Y\rho Y = |-\rangle\langle-| = \tfrac{1}{2}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix},$$

$$Z\rho Z = |-\rangle\langle-|\,.$$

$$\begin{aligned} \mathcal{E}(\rho) &= 0.88\,\rho + 0.04\,(X\rho X) + 0.04\,(Y\rho Y) + 0.04\,(Z\rho Z) \\ &= 0.88\,\rho + 0.04\,\rho + 0.04\,|-\rangle\langle-| + 0.04\,|-\rangle\langle-| = 0.92\,\rho + 0.08\,|-\rangle\langle-|\,. \end{aligned}$$

The off-diagonal coherences shrink: the state has been partially dephased.

2.5 Multi-Qubit Pauli Group

The single-qubit picture extends naturally to n qubits, but two important subtleties arise: general errors become *sums* of tensor products with complex coefficients, and those tensor products must form a multiplicative group, which forces discrete phases $\{+1, -1, +i, -i\}$ into the picture. This subsection explains both points from first principles.

Notation convention. We label qubits $1, 2, \ldots, n$. The subscript on a Pauli operator is *always* the qubit label: P_j means Pauli $P \in \{I, X, Y, Z\}$ acting on qubit j. A tensor-product operator on all n qubits is written $P_1 \otimes P_2 \otimes \cdots \otimes P_n$. In running text and commutation checks we use the shorthand $X_1 Z_2$ to mean $X \otimes Z$. This subscript convention is used consistently throughout.

2.5.1 Tensor products as specific error operators

A single tensor product

$$P_1 \otimes P_2 \otimes \cdots \otimes P_n, \qquad P_j \in \{I, X, Y, Z\},$$

is a **specific error operator**: qubit j experiences exactly error P_j, independently of all other qubits. For example, $X_1 \otimes I_2 \otimes Z_3$ means qubit 1 suffers a bit-flip, qubit 2 is untouched, and qubit 3 suffers a phase-flip, all simultaneously.

By choosing P_j independently at each of the n positions we obtain 4^n distinct tensor-product operators. The space of $2^n \times 2^n$ matrices has dimension $(2^n)^2 = 4^n$. Since the 4^n tensor-product operators are linearly independent, they form a **basis** for the entire operator space — every n-qubit operator can be expanded in this basis.

2.5.2 General errors require a linear combination

Recall the single-qubit decomposition:

$$E = c_0\, I + c_1\, X + c_2\, Y + c_3\, Z, \qquad c_i \in \mathbb{C}.$$

Every basis element is multiplied by its own complex coefficient. The n-qubit case is structurally identical. Introduce the multi-index $\mathbf{k} = (k_1, k_2, \ldots, k_n)$ where $k_j \in \{I, X, Y, Z\}$ specifies which Pauli acts on qubit j. The general n-qubit error is:

$$E \;=\; \sum_{\mathbf{k}} c_{\mathbf{k}} \left(P_{1,k_1} \otimes P_{2,k_2} \otimes \cdots \otimes P_{n,k_n}\right), \qquad c_{\mathbf{k}} \in \mathbb{C},$$

where P_{j,k_j} denotes the Pauli of type k_j acting on qubit j (first subscript = qubit label, second subscript = Pauli type), and the sum runs over all 4^n multi-indices $\mathbf{k}$.

The coefficient $c_{\mathbf{k}}$ plays exactly the same role as c_0, c_1, c_2, c_3 in the single-qubit case: it is a complex number encoding how much of that particular error pattern is present.

Key Point

[Phases are absorbed into $c_{\mathbf{k}}$] Because $c_{\mathbf{k}} \in \mathbb{C}$, it already carries both a magnitude and a phase ($c_{\mathbf{k}} = |c_{\mathbf{k}}|\, e^{i\theta_{\mathbf{k}}}$). Any scalar phase associated with a particular basis element is simply part of $c_{\mathbf{k}}$ and does not need to be written separately. For example, if some physical process contributes $-i$ times the operator $X_1 \otimes Z_2$, then $c_{(X,Z)} = -i$ and the phase $-i$ is encoded inside this coefficient.

The discrete phases $\{+1, -1, +i, -i\}$ that appear in the Pauli *group* (next subsection) are an entirely separate matter: they are fixed values forced by group multiplication, not free continuous parameters chosen to fit an error.

A single tensor product ($c_{\mathbf{k}} = 1$ for one $\mathbf{k}$, zero otherwise) describes a specific, clean error. A *general* noise process is an arbitrary $2^n \times 2^n$ matrix requiring a full weighted sum over all 4^n basis elements. Syndrome measurement collapses this continuous sum onto a single tensor-product term, turning the error into one discrete correctable Pauli.

2.5.3 Why a multiplicative group is necessary

Before introducing the Pauli group, we ask: *why must the Pauli operators form a group at all?*

Syndrome decoding works by checking, for each stabilizer generator g_i, whether the error E commutes or anticommutes with it:

$$g_i E = +E g_i \;\Rightarrow\; \text{syndrome bit } +1, \qquad g_i E = -E g_i \;\Rightarrow\; \text{syndrome bit } -1.$$

This requires *multiplying* Pauli operators together. For the result to be interpretable, it must stay inside the set of operators we are working with. This is the **closure** property of a group, and it is the minimum algebraic requirement for syndrome decoding to be well-defined.

The other group axioms are equally unavoidable:

- **Identity** ($I \otimes \cdots \otimes I$): the "no error" case must belong to the set.
- **Inverses**: every Pauli satisfies $P^2 = I$, so each element is its own inverse — this comes free from the algebra.
- **Associativity**: matrix multiplication is always associative.

Group structure is not imposed by choice; it is forced by the physics of syndrome decoding. The question becomes: does the natural set of tensor-product Paulis already form a group, or must it be enlarged?

2.5.4 Why phases $\{+1, -1, +i, -i\}$ appear

Consider the set of n-qubit tensor products *without* phases:

$$\widetilde{\mathcal{P}}_n = \{P_1 \otimes \cdots \otimes P_n \mid P_j \in \{I, X, Y, Z\}\}.$$

Take $A = X_1 \otimes I_2$ and $B = Y_1 \otimes I_2$ in $\widetilde{\mathcal{P}}_2$:

$$AB = (X \otimes I)(Y \otimes I) = (XY) \otimes I = iZ \otimes I.$$

But $iZ \otimes I \notin \widetilde{\mathcal{P}}_2$ because $iZ \notin \{I, X, Y, Z\}$. **Closure fails**: $\widetilde{\mathcal{P}}_n$ is not a group, so syndrome decoding cannot be defined over it.

The failure arises from the relation $XY = iZ$: multiplying distinct Paulis always produces a phase from $\{+1, -1, +i, -i\}$. The minimal fix is to include all four phases:

Definition 2.1 (n-qubit Pauli group)**.**

$$\mathcal{P}_n = \{\phi \cdot P_1 \otimes \cdots \otimes P_n \mid P_j \in \{I, X, Y, Z\},\ \phi \in \{+1, -1, +i, -i\}\}.$$

Its size is $|\mathcal{P}_n| = 4 \times 4^n = 4^{n+1}$.

Key Point

The phases $\{+1, -1, +i, -i\}$ in $\mathcal{P}_n$ are not physically observable (global phases are undetectable). They are purely algebraic bookkeeping to keep the set closed under multiplication. This group structure powers the stabilizer formalism and makes syndrome decoding well-defined.

Caution

[Group phases versus error-decomposition coefficients] These are two entirely different things:

- **Coefficients** $c_{\mathbf{k}} \in \mathbb{C}$ are *continuous free parameters* representing how much of each basis error is present. Any phase of a basis element is absorbed into $c_{\mathbf{k}}$.
- **Group phases** $\phi \in \{+1, -1, +i, -i\}$ are *discrete fixed values* produced when group elements are multiplied during syndrome calculations. They ensure closure.

Even with phases, $\mathcal{P}_n$ is a *finite discrete set* of 4^{n+1} elements. It cannot represent a general continuous error operator without the continuous coefficients $c_{\mathbf{k}}$. The two concepts operate at completely different levels: coefficients decompose errors; phases maintain group closure.

2.5.5 Phase accumulation when multiplying group elements

Two n-qubit group elements multiply qubit by qubit:

$$(P_1 \otimes \cdots \otimes P_n)(Q_1 \otimes \cdots \otimes Q_n) = (P_1 Q_1) \otimes \cdots \otimes (P_n Q_n).$$

Each single-qubit product $P_j Q_j$ contributes a phase from $\{+1, -1, +i, -i\}$ (from eq. (2)):

$$XY = iZ, \quad YX = -iZ, \quad YZ = iX, \quad ZY = -iX, \quad ZX = iY, \quad XZ = -iY.$$

The overall phase is the product of all n individual phases.

Worked Example 2.4: Phase accumulation in a 3-qubit product

Compute $(X_1 \otimes Y_2 \otimes Z_3)(Y_1 \otimes Z_2 \otimes X_3)$:

$$\begin{aligned} &\text{Qubit 1: } X_1 Y_1 = iZ_1, \quad \text{phase } +i, \\ &\text{Qubit 2: } Y_2 Z_2 = iX_2, \quad \text{phase } +i, \\ &\text{Qubit 3: } Z_3 X_3 = iY_3, \quad \text{phase } +i. \end{aligned}$$

Overall phase: $(+i)^3 = i^3 = -i$. Result: $-i\,(Z_1 \otimes X_2 \otimes Y_3) \in \mathcal{P}_3$. ✓

2.5.6 Do phases matter for error correction?

For *detecting* errors, phases are essential because they determine commutativity. A stabilizer g and an error E either commute ($gE = Eg$, syndrome $+1$) or anticommute ($gE = -Eg$, syndrome -1). The -1 factor distinguishing these cases *is* a phase; without consistent tracking the syndrome calculation is undefined.

For *correcting* errors, global phases on the error do not matter: P, $-P$, and iP cause the same physical damage. The syndrome identifies the Pauli type on each qubit; the correction applies that Pauli regardless of global phase.

2.5.7 Commutation in the multi-qubit Pauli group

Two n-qubit Pauli operators either **commute** ($AB = BA$) or **anticommute** ($AB = -BA$). When there is an odd number of local anticommutations, the minus signs do not cancel, so one overall minus sign remains. Therefore the two multi-qubit operators anticommute.

Worked Example 2.5: Commutation checks in $\mathcal{P}_2$

(a) Do $A = X_1 \otimes Z_2$ and $B = Z_1 \otimes X_2$ commute?
Qubit 1: $X_1 Z_1$ vs $Z_1 X_1$ — anticommute ($XZ = -ZX$); sign -1.
Qubit 2: $Z_2 X_2$ vs $X_2 Z_2$ — anticommute ($ZX = -XZ$); sign -1.
Total sign: $(-1)(-1) = +1$. **They commute.**

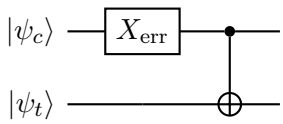
(b) Do $C = X_1 \otimes I_2$ and $D = Z_1 \otimes Z_2$ commute?
Qubit 1: $X_1 Z_1$ vs $Z_1 X_1$ — anticommute; sign -1.
Qubit 2: $I_2 Z_2$ vs $Z_2 I_2$ — commute ($IZ = ZI$); sign $+1$.
Total sign: $(-1)(+1) = -1$. **They anticommute.**

This is the engine of syndrome decoding: stabilizer generators are chosen so that each correctable error anticommutes with a *distinct subset* of them, making the pattern of ± 1 outcomes a unique fingerprint of the error type and location.

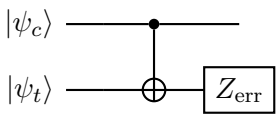
2.6 Error Propagation in Circuits

Errors in gates propagate to other qubits:

- An X error on the *control* of a CNOT propagates to both qubits: $X_c \rightarrow X_c \otimes X_t$.
- A Z error on the *target* of a CNOT propagates to both qubits: $Z_t \rightarrow Z_c \otimes Z_t$.



X error on control $\rightarrow$ X on both



Z error on target $\rightarrow$ Z on both

Figure 2: Error propagation through CNOT.

This motivates careful **fault-tolerant circuit design** covered in Section 8.

2.7 Exercises

Exercise 2.1. Verify eq. (2): compute YZ and ZX by matrix multiplication and confirm they equal iX and iY respectively.

Exercise 2.2. Show that $\{I, X, Y, Z\}$ form a basis for the space of 2×2 complex matrices. (*Hint*: show linear independence and use a dimension argument.)

Exercise 2.3. For the depolarising channel with $p = 0.06$, compute $\mathcal{E}_{\text{dep}}(|0\rangle\langle 0|)$. What is the probability of an X error? Of a Z error?

Exercise 2.4. Determine whether each pair commutes or anticommutes:
(a) $X_1 \otimes X_2$ and $Z_1 \otimes Z_2$; (b) $X_1 \otimes Z_2$ and $X_1 \otimes X_2$; (c) $Y_1 \otimes Y_2$ and $X_1 \otimes Z_2$.

Exercise 2.5. For the amplitude damping channel with decay probability γ, verify that $\mathcal{E}(\rho)$ is a valid density matrix for a general $\rho = \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix}$. What happens to ρ_{11} and ρ_{01} as $\gamma \rightarrow 1$? Interpret physically.

Exercise 2.6. Compute $(Y_1 \otimes X_2)(Z_1 \otimes Y_2)$ in $\mathcal{P}_2$. State the overall phase and the resulting Pauli tensor product.

Section Summary

Section 2 key points:

- $\{I, X, Y, Z\}$ form a basis for all 2×2 complex matrices. Any single-qubit error decomposes as $E = c_0 I + c_1 X + c_2 Y + c_3 Z$ with $c_i \in \mathbb{C}$. Syndrome measurement collapses E to a single Pauli.
- **Kraus operators** $\{K_k\}$ arise by tracing out the environment after joint unitary evolution. Each $K_k = \langle e_k | U | e_0 \rangle$ is the qubit operator for the k-th environmental jump. Completeness $\sum_k K_k^\dagger K_k = I$ follows from unitarity.
- **Notation**: the subscript on a Pauli operator is always the qubit label (P_j = Pauli P on qubit j).
- A tensor product $P_1 \otimes \cdots \otimes P_n$ is a *specific* error (qubit j gets error P_j). All 4^n such products form a basis for n-qubit operator space.
- A *general* n-qubit error requires a linear combination $E = \sum_{\mathbf{k}} c_{\mathbf{k}} (P_{1,k_1} \otimes \cdots \otimes P_{n,k_n})$ with $c_{\mathbf{k}} \in \mathbb{C}$. Any phase of a basis element is absorbed into the complex coefficient $c_{\mathbf{k}}$.
- **Group phases** $\{+1, -1, +i, -i\}$ in $\mathcal{P}_n$ are discrete fixed values forced by closure under multiplication. They are unobservable and entirely distinct from the continuous coefficients $c_{\mathbf{k}}$.
- Two n-qubit Paulis commute iff they anticommute on an even number of positions. This commutation structure drives syndrome decoding.
- Errors propagate through CNOT gates, a key concern for fault-tolerant design.

3 Classical and Quantum Repetition Codes

Intuition

The simplest way to protect a bit is to say it three times. If you tell three people "the answer is **0**" and one of them mishears it as "the answer is **1**", the other two can overrule the mistake. This is a repetition code, and it is the starting point for understanding both classical and quantum error correction.

3.1 Classical 3-Bit Repetition Code

The classical 3-bit repetition code is:

$$0 \longrightarrow 000, \qquad 1 \longrightarrow 111.$$

Decoding by majority vote. If one bit flips, the other two still agree, so majority voting recovers the original bit. A logical error occurs only if two or more bits flip.

Assuming each bit flips independently with probability p:

$$P(\text{logical error}) = \binom{3}{2} p^2(1-p) + \binom{3}{3} p^3$$

$$= 3p^2(1-p) + p^3 = 3p^2 - 2p^3.$$

For small p, this is approximately:

$$P(\text{logical error}) \approx 3p^2.$$

Worked Example 3.1: Classical code improvement

Suppose the physical bit-flip probability is

$$p = 0.01\% = 0.0001.$$

Without encoding, the bit error rate is simply:

$$P(\text{error}) = 0.01\%.$$

With the 3-bit repetition code:

$$P(\text{logical error}) = 3(0.0001)^2 - 2(0.0001)^3$$

$$= 3 \times 10^{-8} - 2 \times 10^{-12} \approx 2.9998 \times 10^{-8}.$$

Expressed as a percentage:

$$P(\text{logical error}) \approx 0.000003\%.$$

So the 3-bit repetition code reduces the error rate by approximately 3333×.

Syndrome table for the classical code. Rather than majority vote, we can identify the error from *parity checks*:

$$s_1 = \text{bit}_1 \oplus \text{bit}_2, \qquad s_2 = \text{bit}_2 \oplus \text{bit}_3.$$

The syndrome (s_1, s_2) tells us exactly which bit (if any) flipped:

Received	**Error**	$s_1 = b_1 \oplus b_2$	$s_2 = b_2 \oplus b_3$
000 (or 111)	None	0	0
100 (or 011)	Flip bit 1	1	0
010 (or 101)	Flip bit 2	1	1
001 (or 110)	Flip bit 3	0	1

3.2 Quantum 3-Qubit Bit-Flip Code

Intuition

We want to protect the quantum state $\alpha|000\rangle + \beta|111\rangle$ from bit-flip errors. The secret is α and β — they must never be revealed or disturbed. Our job is to figure out *which qubit was flipped* without ever peeking at α or β.
We do this by asking two yes/no questions:

1. Do qubits 1 and 2 agree?
2. Do qubits 2 and 3 agree?

The answers together pinpoint the error exactly. Everything else — stabilisers, eigenvalues, commutation rules — is the mathematical language that makes this precise and generalisable.

3.2.1 Encoding

Starting from $|\psi\rangle|0\rangle|0\rangle = (\alpha|0\rangle + \beta|1\rangle)|0\rangle|0\rangle$, two CNOT gates produce the encoded state:

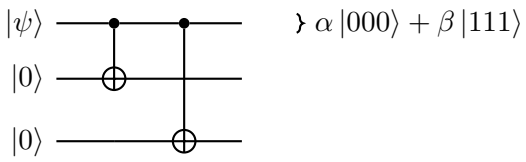


Figure 3: Encoding circuit for the 3-qubit bit-flip code.

Trace through:

$$\begin{aligned}(\alpha|0\rangle + \beta|1\rangle)|00\rangle &\xrightarrow{\text{CNOT}_{1,2}} \alpha|000\rangle + \beta|110\rangle \\ &\xrightarrow{\text{CNOT}_{1,3}} \alpha|000\rangle + \beta|111\rangle\,.\end{aligned}$$

Caution

This is **not** three copies of $|\psi\rangle$ — the no-cloning theorem forbids that. The values α and β live in the *correlations* between the three qubits, not in any single qubit. This is precisely

why we can probe the qubits without disturbing α and β.

3.2.2 What Can Go Wrong

A bit-flip error X on any one qubit changes $|0\rangle \leftrightarrow |1\rangle$. The four possible situations are:

Situation	State after error
No error	$\alpha\,\|000\rangle + \beta\,\|111\rangle$
X_1 flipped qubit 1	$\alpha\,\|100\rangle + \beta\,\|011\rangle$
X_2 flipped qubit 2	$\alpha\,\|010\rangle + \beta\,\|101\rangle$
X_3 flipped qubit 3	$\alpha\,\|001\rangle + \beta\,\|110\rangle$

In every case the three qubits are either all the same (no error) or one of them disagrees with the other two (error). The two yes/no questions above perfectly distinguish all four cases.

3.2.3 The Syndrome Circuit

We use two ancilla qubits, each starting in $|0\rangle$, to record the answers to our two questions.

Key Point

A CNOT gate with a **data qubit as control** and the ancilla as target copies the bit value of the data qubit into the ancilla. Two CNOTs in a row leave the ancilla holding the **XOR (parity)** of the two data qubits. That parity is exactly the answer to our yes/no question.

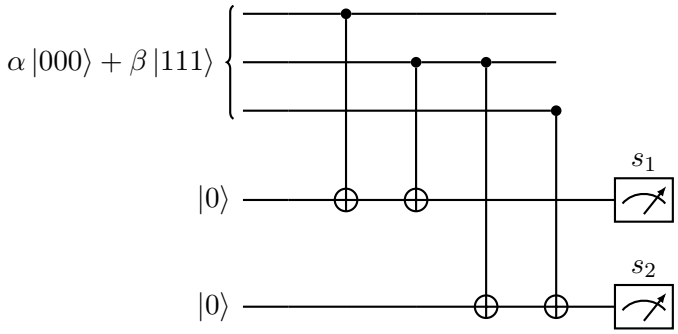


Figure 4: Syndrome measurement circuit for the 3-qubit bit-flip code. Ancilla 1 measures the parity of qubits 1 and 2 (giving s_1); ancilla 2 measures the parity of qubits 2 and 3 (giving s_2). The measurement is completely standard — just read out 0 or 1. The values α and β are never disturbed.

The measurement outcomes $s_1, s_2 \in \{0, 1\}$ are read by the following table:

Error	State after error	s_1	s_2	Correction
None	$\alpha\|000\rangle + \beta\|111\rangle$	0	0	None
X_1	$\alpha\|100\rangle + \beta\|011\rangle$	1	0	Apply X_1
X_2	$\alpha\|010\rangle + \beta\|101\rangle$	1	1	Apply X_2
X_3	$\alpha\|001\rangle + \beta\|110\rangle$	0	1	Apply X_3

Figure 5: Syndrome table using measurement outcomes $s_1, s_2 \in \{0, 1\}$. An outcome of 0 means the two qubits agree; 1 means they disagree.

3.2.4 The Stabiliser Language: Z_1Z_2 and Eigenvalues

The circuit above is complete and correct on its own. But physicists write the same information using the operator Z_1Z_2 and speak of eigenvalues $+1$ and -1. Here is exactly what that means.

Recall that the Z gate has eigenstates $|0\rangle$ and $|1\rangle$:

$$Z|0\rangle = +1 \cdot |0\rangle, \qquad Z|1\rangle = -1 \cdot |1\rangle.$$

So measuring whether a qubit is $|0\rangle$ or $|1\rangle$ is the same as asking: is the Z eigenvalue $+1$ or -1?

The operator Z_1Z_2 applies Z to qubit 1 and Z to qubit 2, and multiplies the two eigenvalues together:

Qubits 1,2	Z_1 eigenvalue	Z_2 eigenvalue	Z_1Z_2 eigenvalue	Measurement s_1
$\|00\rangle$	$+1$	$+1$	$+\mathbf{1}$ (agree)	0
$\|11\rangle$	-1	-1	$+\mathbf{1}$ (agree)	0
$\|01\rangle$	$+1$	-1	$-\mathbf{1}$ (disagree)	1
$\|10\rangle$	-1	$+1$	$-\mathbf{1}$ (disagree)	1

The mapping is simply:

$$s_1 = 0 \longleftrightarrow Z_1Z_2 \text{ eigenvalue} = +1, \qquad s_1 = 1 \longleftrightarrow Z_1Z_2 \text{ eigenvalue} = -1.$$

The operators $g_1 = Z_1Z_2$ and $g_2 = Z_2Z_3$ are called **stabilisers** of the code: they both give eigenvalue $+1$ on every error-free codeword, and flip to -1 when the right error occurs.

3.2.5 Commutation and Anticommutation: Will an Error Be Caught?

The stabiliser language gives us a powerful shortcut. Instead of tracing through the circuit every time, we can answer the question *"will this error be detected?"* with a single algebraic check.

Key Point

[The golden rule of error detection]

- If an error E **anticommutes** with a stabiliser g (i.e. $Eg = -gE$), then the syndrome bit for g **flips** from 0 to 1 $\Rightarrow$ error **detected**.

- If an error E **commutes** with a stabiliser g (i.e. $Eg = gE$), then the syndrome bit for g **stays 0** $\Rightarrow$ error **invisible** to g.

We already know how Pauli gates interact on the *same* qubit:

$$XZ = -ZX \quad \text{(anticommute)}, \qquad ZZ = ZZ \quad \text{(commute)}.$$

Operators acting on *different* qubits always commute — they do not interact.

Worked Example 3.2: X_1 error: will it be caught?

Check against $g_1 = Z_1Z_2$: X_1 acts on qubit 1; Z_1Z_2 also acts on qubit 1 via Z_1. Since X and Z anticommute on the same qubit:

$$X_1 \cdot (Z_1Z_2) = (X_1Z_1)Z_2 = -(Z_1X_1)Z_2 = -(Z_1Z_2) \cdot X_1.$$

X_1 **anticommutes** with $g_1 \Rightarrow s_1$ flips to 1.
Check against $g_2 = Z_2Z_3$: X_1 acts on qubit 1 only; Z_2Z_3 acts on qubits 2 and 3 only. Different qubits $\Rightarrow$ they commute:

$$X_1 \cdot (Z_2Z_3) = (Z_2Z_3) \cdot X_1.$$

X_1 **commutes** with $g_2 \Rightarrow s_2$ stays 0.
Result: $(s_1, s_2) = (1, 0) \Rightarrow$ qubit 1 is identified and corrected. ✓

Worked Example 3.3: X_2 error: will it be caught?

X_2 shares qubit 2 with both $g_1 = Z_1Z_2$ and $g_2 = Z_2Z_3$. It anticommutes with both:

$$X_2 \cdot (Z_1Z_2) = Z_1(X_2Z_2) = Z_1(-Z_2X_2) = -(Z_1Z_2) \cdot X_2,$$
$$X_2 \cdot (Z_2Z_3) = (X_2Z_2)Z_3 = (-Z_2X_2)Z_3 = -(Z_2Z_3) \cdot X_2.$$

Both syndrome bits flip.
Result: $(s_1, s_2) = (1, 1) \Rightarrow$ qubit 2 is identified and corrected. ✓

Worked Example 3.4: Z_1 error: will it be caught?

This is the revealing case. Does Z_1 commute or anticommute with $g_1 = Z_1Z_2$? Z commutes with itself on the same qubit ($ZZ = I$):

$$Z_1 \cdot (Z_1Z_2) = (Z_1Z_1)Z_2 = IZ_2 = Z_2 = (Z_1Z_2) \cdot Z_1.$$

Z_1 **commutes** with $g_1 \Rightarrow s_1$ stays 0.
Z_1 also commutes with $g_2 = Z_2Z_3$ (they act on different qubits) $\Rightarrow s_2$ stays 0.
Result: $(s_1, s_2) = (0, 0)$ — the syndrome says "no error," but the phase-flip Z_1 *did* happen and is completely **undetected**.

Caution

The 3-qubit bit-flip code is **blind to all Z (phase-flip) errors**. This is not a bug in the circuit — it is a fundamental consequence of the algebra: every Z error commutes with every stabiliser Z_iZ_j, so no syndrome bit ever flips. Detecting phase errors requires a different code with different stabilisers.

The complete picture, combining both the 0/1 measurement language and the eigenvalue language, is shown below.

Error	State after error	$g_1 = Z_1Z_2$	$g_2 = Z_2Z_3$	Correction
None	$\alpha\|000\rangle + \beta\|111\rangle$	$+1$	$+1$	None
X_1	$\alpha\|100\rangle + \beta\|011\rangle$	-1	$+1$	Apply X_1
X_2	$\alpha\|010\rangle + \beta\|101\rangle$	-1	-1	Apply X_2
X_3	$\alpha\|001\rangle + \beta\|110\rangle$	$+1$	-1	Apply X_3

Figure 6: Syndrome table in the eigenvalue language. $g_i = +1$ corresponds to measurement outcome $s_i = 0$ (qubits agree); $g_i = -1$ corresponds to $s_i = 1$ (qubits disagree). The syndrome uniquely identifies single-qubit bit-flip errors. α and β never appear.

3.2.6 Full Syndrome Computation: Worked Example

Worked Example 3.5: Full syndrome computation for X_2

Suppose qubit 2 suffers a bit-flip. The error X_2 gives:

$$X_2\big(\alpha\,|000\rangle + \beta\,|111\rangle\big) = \alpha\,|010\rangle + \beta\,|101\rangle\,.$$

Measuring $g_1 = Z_1Z_2$:

$$Z_1Z_2\,|010\rangle = Z\,|0\rangle \otimes Z\,|1\rangle \otimes I\,|0\rangle = (+1)(-1)\,|010\rangle = -\,|010\rangle\,,$$
$$Z_1Z_2\,|101\rangle = Z\,|1\rangle \otimes Z\,|0\rangle \otimes I\,|1\rangle = (-1)(+1)\,|101\rangle = -\,|101\rangle\,.$$

Both terms give eigenvalue -1, so $g_1 = -1$ (i.e. $s_1 = 1$).
Measuring $g_2 = Z_2Z_3$:

$$Z_2Z_3\,|010\rangle = Z\,|1\rangle \otimes Z\,|0\rangle = (-1)(+1) = -1,$$
$$Z_2Z_3\,|101\rangle = Z\,|0\rangle \otimes Z\,|1\rangle = (+1)(-1) = -1.$$

So $g_2 = -1$ (i.e. $s_2 = 1$).
Syndrome: $(g_1, g_2) = (-1, -1)$, equivalently $(s_1, s_2) = (1, 1)$. From the table, this is an X_2 error. Apply X_2 to correct:

$$X_2\big(\alpha\,|010\rangle + \beta\,|101\rangle\big) = \alpha\,|000\rangle + \beta\,|111\rangle\,. \quad \checkmark$$

The encoded state is perfectly restored. Note that α and β were never measured and are unaffected.

3.2.7 Why Write Z_1Z_2 at All?

For this simple three-qubit code you can just use the 0/1 table and never think about Z_1Z_2. The operator notation pays off when codes grow large. For a surface code with 1000 qubits and 500 stabilisers, tracing through every circuit is impractical. With the commutation algebra you can instantly answer:

- *Will this error be detected?* Check whether it anticommutes with any stabiliser.

- *Will measuring this stabiliser disturb the data?* Check whether the stabiliser commutes with the logical operators.
- *Can two stabilisers be measured independently?* Check whether they commute with each other.

Two symbols and a sign give you all of that without drawing a single circuit.

Section Summary

Section 3.2 key points:

- The 3-qubit bit-flip code encodes $\alpha|0\rangle + \beta|1\rangle$ as $\alpha|000\rangle + \beta|111\rangle$. This is *not* cloning; α, β live in correlations.
- Two ancilla qubits measure the parities of adjacent qubit pairs via CNOT gates and standard 0/1 measurements — nothing exotic.
- The measurement outcomes $s_1, s_2 \in \{0, 1\}$ are equivalent to the stabiliser eigenvalues: $s_i = 0 \leftrightarrow g_i = +1$, $s_i = 1 \leftrightarrow g_i = -1$.
- **Golden rule:** an error is detected if and only if it *anticommutes* with at least one stabiliser.
- X errors anticommute with Z_1Z_2 or Z_2Z_3 (or both) $\Rightarrow$ always detected and corrected.
- Z errors commute with both stabilisers $\Rightarrow$ completely invisible to this code.
- A code that corrects both X and Z errors requires new stabilisers — this is the Shor 9-qubit code (Section 4).

3.3 Limitations: Phase Errors Are Not Corrected

The 3-qubit bit-flip code protects against X errors but is *blind* to Z errors.

Worked Example 3.6: Phase flip on qubit 1 is undetected

Suppose a phase-flip Z_1 occurs on the encoded state:

$$Z_1\big(\alpha|000\rangle + \beta|111\rangle\big) = \alpha|000\rangle - \beta|111\rangle\,.$$

Check the syndrome:

$$\begin{aligned} Z_1Z_2(\alpha|000\rangle - \beta|111\rangle) &= \alpha(+1)(+1)|000\rangle - \beta(-1)(-1)|111\rangle \\ &= \alpha|000\rangle - \beta|111\rangle\,, \quad g_1 = +1, \\ Z_2Z_3(\alpha|000\rangle - \beta|111\rangle) &= +\alpha|000\rangle - \beta|111\rangle\,, \quad g_2 = +1. \end{aligned}$$

Syndrome $(+1, +1)$ says "no error" — but the logical state has been changed from $\alpha|0\rangle_L + \beta|1\rangle_L$ to $\alpha|0\rangle_L - \beta|1\rangle_L$. The error is *undetected*.

3.4 Quantum Phase-Flip Code

To correct phase errors, we move to the *Hadamard basis* $\{|+\rangle, |-\rangle\}$:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

In this basis, Z errors become X errors: $Z|+\rangle = |-\rangle$ and $Z|-\rangle = |+\rangle$.

Encode the logical qubit as:

$$|0\rangle_L = |+++\rangle\,, \qquad |1\rangle_L = |---\rangle\,,$$

so $|\psi\rangle_L = \alpha|+++\rangle + \beta|---\rangle$.

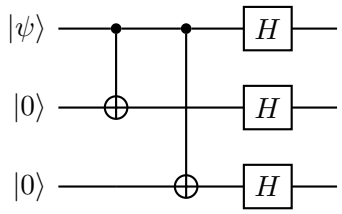


Figure 7: Phase-flip code encoding: first encode with CNOTs (as in the bit-flip code), then apply Hadamard to each qubit.

The stabilizers are now X_1X_2 and X_2X_3 (parity in the Hadamard basis), detecting Z errors. The decoding applies $H^{\otimes 3}$ to convert back, then corrects as before.

Caution

The phase-flip code corrects Z errors only; the bit-flip code corrects X errors only. Neither code corrects both types simultaneously. For complete single-qubit error correction, we need a code that handles both — this is the **Shor 9-qubit code** (Section 4).

3.5 Why This Works Despite No-Cloning

A common confusion: if we cannot clone $|\psi\rangle$, how can we store it in three qubits? The key is that the encoded state $\alpha|000\rangle + \beta|111\rangle$ is *not* three copies of $|\psi\rangle$.

- Tracing out any one qubit of $\alpha|000\rangle + \beta|111\rangle$ gives a *mixed* state (not $|\psi\rangle$).
- The information about α and β is in the *correlations* between the three qubits, not in any single qubit.
- Syndrome measurement reveals only the *parity* of pairs, not α or β individually.

Exercise 3.1. Verify that the syndrome $(g_1, g_2) = (+1, -1)$ correctly identifies an X_3 error by carrying out the eigenvalue calculation explicitly.

Exercise 3.2. Show that the 3-qubit bit-flip code has *two* possible corrective operations for each non-trivial syndrome — for instance, when $(g_1, g_2) = (-1, +1)$ you might apply X_1. Why does it not matter whether we apply X_1 or instead apply X_2X_3 (which has the same syndrome)? (*Hint:* what is the product $X_1 \cdot X_2X_3$ on the encoded subspace?)

Exercise 3.3. Derive the encoding circuit for the quantum phase-flip code by: (a) showing that $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$; (b) tracing $|\psi\rangle|0\rangle|0\rangle$ through the CNOT+Hadamard circuit to confirm $|0\rangle_L = |+++\rangle$ and $|1\rangle_L = |---\rangle$.

Exercise 3.4. Why does the 3-qubit bit-flip code fail to correct a Y error on qubit 2? Find the syndrome and show that applying the "correction" actually introduces an additional error.

Section Summary

Section 3 key points:

- The quantum 3-qubit bit-flip code encodes $\alpha|0\rangle + \beta|1\rangle$ as $\alpha|000\rangle + \beta|111\rangle$. This is *not* cloning; α, β live in correlations.
- Syndrome measurement identifies which qubit was flipped without disturbing the logical information.
- This code protects against X errors *only*; it is blind to Z (phase) errors.
- The phase-flip code encodes in the Hadamard basis, protecting against Z errors only.
- A code that corrects both X and Z errors requires a new idea: the Shor code.

4 The Shor 9-Qubit Code

Intuition

The **3-qubit bit-flip code** can fix X (bit-flip) errors but is helpless against Z (phase-flip) errors. Conversely, the **3-qubit phase-flip code** fixes Z errors but not X errors. Peter Shor's key insight was to *combine both codes by nesting one inside the other* — a technique called **concatenation**:

1. **Outer layer (phase-flip code):** Encode the single logical qubit into *three* qubits using the phase-flip code. This protects against Z errors across the three groups.
2. **Inner layer (bit-flip code):** Take each of those three qubits and encode *it* into three more qubits using the bit-flip code. This protects against X errors within each group.

The result uses $3\times3 = 9$ physical qubits to store 1 logical qubit, and corrects *any* single-qubit error (X, Z, or Y) on any one of the 9 qubits.

4.1 Step-by-Step Encoding

It is easiest to understand the encoding in two stages.

Stage 1 — Phase-flip code (outer layer)

Start with an arbitrary logical qubit $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$. The 3-qubit phase-flip code maps:

$$\alpha\,|0\rangle + \beta\,|1\rangle \xrightarrow{\text{phase-flip encode}} \alpha\,|+\rangle\,|+\rangle\,|+\rangle + \beta\,|-\rangle\,|-\rangle\,|-\rangle\,,$$

where $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.

Expanding $|+\rangle$ and $|-\rangle$ explicitly:

$$\begin{aligned}\alpha\,|+\rangle\,|+\rangle\,|+\rangle + \beta\,|-\rangle\,|-\rangle\,|-\rangle = {}& \frac{\alpha}{(\sqrt{2})^3}\big(|0\rangle + |1\rangle\big)\big(|0\rangle + |1\rangle\big)\big(|0\rangle + |1\rangle\big) \\ & + \frac{\beta}{(\sqrt{2})^3}\big(|0\rangle - |1\rangle\big)\big(|0\rangle - |1\rangle\big)\big(|0\rangle - |1\rangle\big).\end{aligned}$$

At this point we have **3 qubits**, one per block.

Stage 2 — Bit-flip code (inner layer)

Now apply the 3-qubit bit-flip code independently to each of the 3 qubits from Stage 1:

$$|0\rangle \longrightarrow |000\rangle\,, \qquad |1\rangle \longrightarrow |111\rangle\,.$$

Replacing every $|0\rangle$ with $|000\rangle$ and every $|1\rangle$ with $|111\rangle$ inside the Stage-1 expression gives the final 9-qubit codewords. The logical codewords (subscript L stands for **logical**, distinguishing these 9-qubit encoded states from single physical qubits; $|0\rangle_L$ and $|1\rangle_L$ are *not* individual qubits but entire 9-qubit states that an algorithm treats as a single qubit) are:

$$|0\rangle_L = \frac{1}{2\sqrt{2}}\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big), \tag{4}$$

$$|1\rangle_L = \frac{1}{2\sqrt{2}}\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big). \tag{5}$$

Key Point

The normalisation factor $\frac{1}{2\sqrt{2}}$ comes from $\left(\frac{1}{\sqrt{2}}\right)^3 = \frac{1}{2\sqrt{2}}$. Each block $\left(|000\rangle \pm |111\rangle\right)$ is not individually normalised; the factor ensures the full 9-qubit state has unit norm.

A general logical state $|\psi\rangle_L = \alpha\,|0\rangle_L + \beta\,|1\rangle_L$ is therefore:

$$|\psi\rangle_L = \frac{\alpha}{2\sqrt{2}}\left(|000\rangle + |111\rangle\right)^{\otimes 3} + \frac{\beta}{2\sqrt{2}}\left(|000\rangle - |111\rangle\right)^{\otimes 3}.$$

4.1.1 Block structure

The 9 qubits are divided into **three blocks of three**:

$$\underbrace{q_1\,q_2\,q_3}_{\text{Block 1}},\quad \underbrace{q_4\,q_5\,q_6}_{\text{Block 2}},\quad \underbrace{q_7\,q_8\,q_9}_{\text{Block 3}}.$$

Within each block a bit-flip code is active; across the three blocks a phase-flip code is active. Figure 8 shows this nested layout.

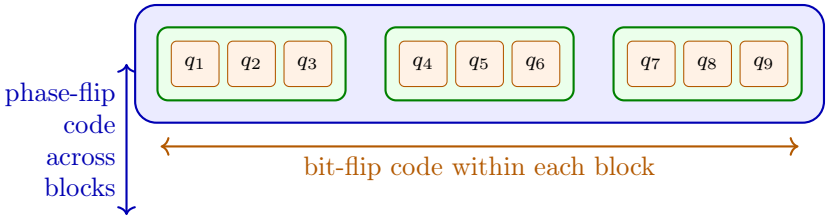


Figure 8: Nested structure of the Shor 9-qubit code. The three inner green boxes each form a 3-qubit *bit-flip* code protecting against X errors. The outer blue box represents the 3-qubit *phase-flip* code protecting against Z errors across the three blocks.

4.1.2 Encoding Circuit and What It Produces

Figure 9 shows the encoding circuit. We trace through it step by step for input $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$.

Initial state.

$$|\Psi_0\rangle = \left(\alpha\,|0\rangle + \beta\,|1\rangle\right) \otimes |0\rangle^{\otimes 8}.$$

Step 1 — Two CNOTs copy q_1 to q_4 and q_7. Qubits q_4 and q_7 become the "leaders" of blocks 2 and 3:

$$|\Psi_1\rangle = \alpha\,|0\rangle\,|00\rangle\,|0\rangle\,|00\rangle\,|0\rangle\,|00\rangle + \beta\,|1\rangle\,|00\rangle\,|1\rangle\,|00\rangle\,|1\rangle\,|00\rangle.$$

(Within each block, only the leader qubit is set; the two ancillas are still $|0\rangle$.)

Step 2 — Hadamard on each block leader q_1, q_4, q_7. Each leader transforms as $|0\rangle \to |+\rangle$ and $|1\rangle \to |-\rangle$:

$$\begin{aligned}|\Psi_2\rangle &= \alpha\,|+\rangle\,|00\rangle\,|+\rangle\,|00\rangle\,|+\rangle\,|00\rangle + \beta\,|-\rangle\,|00\rangle\,|-\rangle\,|00\rangle\,|-\rangle\,|00\rangle \\ &= \frac{\alpha}{2\sqrt{2}}\left(|0\rangle + |1\rangle\right)|00\rangle\,\left(|0\rangle + |1\rangle\right)|00\rangle\,\left(|0\rangle + |1\rangle\right)|00\rangle \\ &\quad + \frac{\beta}{2\sqrt{2}}\left(|0\rangle - |1\rangle\right)|00\rangle\,\left(|0\rangle - |1\rangle\right)|00\rangle\,\left(|0\rangle - |1\rangle\right)|00\rangle.\end{aligned}$$

Step 3 — Two CNOTs within each block spread the leader to its two ancillas. Inside each block: $|0\rangle|00\rangle \to |000\rangle$ and $|1\rangle|00\rangle \to |111\rangle$. The full 9-qubit output state is:

$$|\Psi_{\text{enc}}\rangle = \frac{\alpha}{2\sqrt{2}}\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big)\big(|000\rangle + |111\rangle\big) + \frac{\beta}{2\sqrt{2}}\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big)\big(|000\rangle - |111\rangle\big) \tag{6}$$

which is exactly $\alpha|0\rangle_L + \beta|1\rangle_L$ as required.

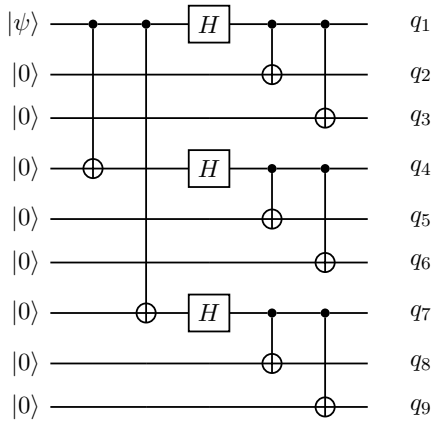


Figure 9: Shor code encoding circuit. **Step 1** (columns 1–2): two CNOTs copy the logical qubit to q_4 and q_7, forming the phase-flip backbone across the three blocks. **Step 2** (column 3): Hadamards on q_1, q_4, q_7 create $(\pm)$-basis superpositions in each block leader. **Step 3** (columns 4–5): CNOTs within each block spread the leader to its two ancillas, producing the $|000\rangle \pm |111\rangle$ structure. Output state: Eq. (6).

4.2 Stabilizer Generators

The Shor code is a **stabilizer code** (see Section 6). A stabilizer is an operator g such that $g|\psi_L\rangle = +|\psi_L\rangle$ for every valid codeword. Errors are detected because an error E on a single qubit will *anticommute* with some stabilizer generators, flipping their eigenvalue from $+1$ to -1.

The 8 independent generators divide into two families.

4.2.1 Bit-flip stabilizers (within each block)

Each block of three qubits has exactly **two** bit-flip stabilizers — one for each adjacent pair:

Block	Qubits	Stabilizers	
Block 1	q_1, q_2, q_3	$g_1 = Z_1Z_2,$	$g_2 = Z_2Z_3$
Block 2	q_4, q_5, q_6	$g_3 = Z_4Z_5,$	$g_4 = Z_5Z_6$
Block 3	q_7, q_8, q_9	$g_5 = Z_7Z_8,$	$g_6 = Z_8Z_9$

Intuition

Within each block the valid states are $|000\rangle$ and $|111\rangle$. Both satisfy $Z_iZ_j = +1$ (the two qubits always agree). An X error on any qubit breaks that agreement and flips the eigenvalue to -1 for the stabilizers that include that qubit. Each pair of stabilizers within a block behaves exactly like the 3-qubit bit-flip code from Section 3.2.

4.2.2 Phase-flip stabilizers (across blocks)

Two stabilizers compare the *relative phase* between blocks:

$$g_7 = X_1X_2X_3X_4X_5X_6, \qquad g_8 = X_1X_2X_3X_7X_8X_9.$$

Intuition

Think of g_7 as asking: "does block 1 have the same phase as block 2?" and g_8 as asking: "does block 1 have the same phase as block 3?" In a valid codeword all three blocks agree, so both give $+1$. A Z error anywhere in a block flips that block's phase, which shows up as a -1 in whichever of g_7, g_8 involves that block.
Note that g_7 and g_8 act on all six and six qubits respectively, but they only care about the *block-level* phase — the same job X_1X_2 did in the 3-qubit phase-flip code, now scaled up.

Key Point

Eight generators for 9 physical qubits $\Rightarrow 2^{9-8} = 2$ code states, encoding exactly **1 logical qubit**. All 8 generators commute pairwise — a necessary condition for a valid stabilizer code.

4.3 Error Correction: The Key Shortcut

Key Point

[You never need to look at all 8 stabilizers at once] The 8 stabilizers split into two completely independent jobs:

- g_1, g_2 tell you about bit-flip errors **in block 1 only**.
- g_3, g_4 tell you about bit-flip errors **in block 2 only**.
- g_5, g_6 tell you about bit-flip errors **in block 3 only**.
- g_7, g_8 tell you about phase-flip errors **across blocks**.

In practice you read the two phase stabilizers first to find out *which block* (if any) has a phase error, and the two bit-flip stabilizers for *each block* to find out which qubit (if any) was bit-flipped. Each pair of stabilizers is decoded exactly like the 3-qubit syndrome tables you already know.

4.3.1 Step 1 — Find bit-flip errors using the block tables

Each block is decoded using the same table as the 3-qubit bit-flip code. You read g_{2k-1} and g_{2k} for block k and look up the result:

g_{2k-1}	g_{2k}	**Meaning**	**Correction**
$+1$	$+1$	No bit-flip in this block	None
-1	$+1$	First qubit of block flipped	Apply X to qubit 1 of block
-1	-1	Middle qubit of block flipped	Apply X to qubit 2 of block
$+1$	-1	Last qubit of block flipped	Apply X to qubit 3 of block

Applied to the three blocks explicitly:

g_1	g_2	Error in Block 1	Fix
$+1$	$+1$	None	—
-1	$+1$	X_1	Apply X_1
-1	-1	X_2	Apply X_2
$+1$	-1	X_3	Apply X_3

g_3	g_4	Error in Block 2	Fix
$+1$	$+1$	None	—
-1	$+1$	X_4	Apply X_4
-1	-1	X_5	Apply X_5
$+1$	-1	X_6	Apply X_6

g_5	g_6	Error in Block 3	Fix
$+1$	$+1$	None	—
-1	$+1$	X_7	Apply X_7
-1	-1	X_8	Apply X_8
$+1$	-1	X_9	Apply X_9

Intuition

Each table is literally the same table as Section 3.2. You have just run the same small 3-qubit syndrome decoder three times, once per block, completely independently.

4.3.2 Step 2 — Find phase-flip errors using the outer table

The two phase stabilizers g_7 and g_8 play exactly the same role as the two stabilizers of the 3-qubit phase-flip code, but now acting on whole blocks rather than individual qubits.

g_7	g_8	Block with phase error	Fix
$+1$	$+1$	None	—
-1	-1	Block 1 (q_1, q_2, q_3)	Apply Z to any qubit in block 1
-1	$+1$	Block 2 (q_4, q_5, q_6)	Apply Z to any qubit in block 2
$+1$	-1	Block 3 (q_7, q_8, q_9)	Apply Z to any qubit in block 3

Intuition

A Z error on *any* qubit within a block flips that entire block's phase relative to the others. The syndrome only tells you *which block* is affected, not which specific qubit within it — but that is fine, because Z errors on different qubits within the same block all have the same effect on the encoded state. Applying Z to any one qubit of that block undoes the damage.

4.3.3 Worked Example: X_5 bit-flip on qubit 5

Worked Example 4.1: Correcting a bit-flip (X_5) on qubit 5

Qubit 5 is the middle qubit of block 2.

Block tables (bit-flip check):

Stabilizer	Shares qubit with X_5?	Outcome
$g_1 = Z_1Z_2$	No (block 1)	$+1$
$g_2 = Z_2Z_3$	No (block 1)	$+1$
$g_3 = Z_4Z_5$	Yes (Z_5 anticommutes with X_5)	-1
$g_4 = Z_5Z_6$	Yes (Z_5 anticommutes with X_5)	-1
$g_5 = Z_7Z_8$	No (block 3)	$+1$
$g_6 = Z_8Z_9$	No (block 3)	$+1$

Only block 2 fired. Look up $(g_3, g_4) = (-1, -1)$ in the block 2 table $\Rightarrow$ middle qubit of block 2 $\Rightarrow$ **qubit 5**.

Phase check:
X_5 commutes with every X in g_7 and g_8 ($X_5 \cdot X_5 = I$, same qubit; X_5 does not appear in g_8 at all).

Stabilizer	Outcome
$g_7 = X_1X_2X_3X_4X_5X_6$	$+1$
$g_8 = X_1X_2X_3X_7X_8X_9$	$+1$

$(g_7, g_8) = (+1, +1) \Rightarrow$ no phase error.

Full syndrome: $(g_1, \ldots, g_8) = (+, +, -, -, +, +, +, +)$.

Correction: Apply X_5. ✓

4.3.4 Worked Example: Z_2 phase-flip on qubit 2

Worked Example 4.2: Correcting a phase-flip (Z_2) on qubit 2

Qubit 2 is inside block 1.

Block tables (bit-flip check):
Z errors commute with Z-type stabilizers ($ZZ = ZZ$ on the same qubit). So all six bit-flip stabilizers give $+1$:

Stabilizer	Outcome
$g_1 = Z_1Z_2$	$+1$ (Z_2 commutes with Z_2)
$g_2 = Z_2Z_3$	$+1$ (Z_2 commutes with Z_2)
g_3, g_4, g_5, g_6	$+1$ (different block entirely)

No bit-flip anywhere.

Phase check:
Z_2 anticommutes with X_2. Qubit 2 appears in both g_7 and g_8:

Stabilizer	Contains X_2?	Outcome
$g_7 = X_1X_2X_3X_4X_5X_6$	Yes $\Rightarrow$ anticommutes	-1
$g_8 = X_1X_2X_3X_7X_8X_9$	Yes $\Rightarrow$ anticommutes	-1

Look up $(g_7, g_8) = (-1, -1)$ in the outer table $\Rightarrow$ **block 1** has a phase error.

Full syndrome: $(g_1, \ldots, g_8) = (+, +, +, +, +, +, -, -)$.

Correction: Apply Z to any qubit in block 1, e.g. Z_1.

Note: The syndrome tells us block 1 has a phase error but not which specific qubit caused it. This is fine — a Z error on q_1, q_2, or q_3 all produce the same block-level syndrome and all require the same fix (Z on any qubit of block 1). ✓

4.3.5 Correcting Y errors

A Y error is just X and Z happening simultaneously ($Y = iXZ$). It fires *both* the bit-flip and the phase-flip syndromes at once.

Worked Example 4.3: Correcting Y_5 on qubit 5

$Y_5 = iX_5Z_5$.
X_5 fires the block 2 bit-flip syndrome: $(g_3, g_4) = (-1, -1) \Rightarrow$ qubit 5 bit-flipped.
Z_5 is inside block 2, so it fires g_7 (which contains X_5) but not g_8 (which does not): $(g_7, g_8) = (-1, +1) \Rightarrow$ block 2 phase-flipped.
Correction: Apply X_5 (fix the bit-flip) and Z_5 (fix the phase-flip), which is equivalent to applying Y_5. ✓

4.3.6 Complete Syndrome Reference Table

The table below lists the syndrome for every possible single-qubit Pauli error. Reading it confirms that all $3 \times 9 = 27$ non-trivial error syndromes are distinct.

Error	g_1	g_2	g_3	g_4	g_5	g_6	g_7	g_8	Correction
None	$+$	$+$	$+$	$+$	$+$	$+$	$+$	$+$	—
X_1	$-$	$+$	$+$	$+$	$+$	$+$	$+$	$+$	Apply X_1
X_2	$-$	$-$	$+$	$+$	$+$	$+$	$+$	$+$	Apply X_2
X_3	$+$	$-$	$+$	$+$	$+$	$+$	$+$	$+$	Apply X_3
X_4	$+$	$+$	$-$	$+$	$+$	$+$	$+$	$+$	Apply X_4
X_5	$+$	$+$	$-$	$-$	$+$	$+$	$+$	$+$	Apply X_5
X_6	$+$	$+$	$+$	$-$	$+$	$+$	$+$	$+$	Apply X_6
X_7	$+$	$+$	$+$	$+$	$-$	$+$	$+$	$+$	Apply X_7
X_8	$+$	$+$	$+$	$+$	$-$	$-$	$+$	$+$	Apply X_8
X_9	$+$	$+$	$+$	$+$	$+$	$-$	$+$	$+$	Apply X_9
Z_1	$+$	$+$	$+$	$+$	$+$	$+$	$-$	$-$	Apply Z_1
Z_2	$+$	$+$	$+$	$+$	$+$	$+$	$-$	$-$	Apply Z_1
Z_3	$+$	$+$	$+$	$+$	$+$	$+$	$-$	$-$	Apply Z_1
Z_4	$+$	$+$	$+$	$+$	$+$	$+$	$-$	$+$	Apply Z_4
Z_5	$+$	$+$	$+$	$+$	$+$	$+$	$-$	$+$	Apply Z_4
Z_6	$+$	$+$	$+$	$+$	$+$	$+$	$-$	$+$	Apply Z_4
Z_7	$+$	$+$	$+$	$+$	$+$	$+$	$+$	$-$	Apply Z_7
Z_8	$+$	$+$	$+$	$+$	$+$	$+$	$+$	$-$	Apply Z_7
Z_9	$+$	$+$	$+$	$+$	$+$	$+$	$+$	$-$	Apply Z_7
Y_1	$-$	$+$	$+$	$+$	$+$	$+$	$-$	$-$	Apply Y_1
Y_2	$-$	$-$	$+$	$+$	$+$	$+$	$-$	$-$	Apply Y_2
Y_3	$+$	$-$	$+$	$+$	$+$	$+$	$-$	$-$	Apply Y_3
Y_4	$+$	$+$	$-$	$+$	$+$	$+$	$-$	$+$	Apply Y_4
Y_5	$+$	$+$	$-$	$-$	$+$	$+$	$-$	$+$	Apply Y_5
Y_6	$+$	$+$	$+$	$-$	$+$	$+$	$-$	$+$	Apply Y_6
Y_7	$+$	$+$	$+$	$+$	$-$	$+$	$+$	$-$	Apply Y_7
Y_8	$+$	$+$	$+$	$+$	$-$	$-$	$+$	$-$	Apply Y_8
Y_9	$+$	$+$	$+$	$+$	$+$	$-$	$+$	$-$	Apply Y_9

Key Point

Notice that Z_1, Z_2, and Z_3 all give the same syndrome $(g_7, g_8) = (-1, -1)$ with all bit-flip stabilizers at $+1$. This is expected and fine: all three errors have the same effect on the encoded state, and applying Z to any one qubit in block 1 corrects all of them. The same applies within block 2 and block 3.

Every Y error produces a syndrome that is exactly the *combination* of the corresponding X and Z syndromes — confirming the two layers of the code work completely independently.

4.4 Correcting Arbitrary Single-Qubit Errors

Any single-qubit error on any of the 9 qubits can be written as $E = c_0 I + c_1 X + c_2 Y + c_3 Z$. Because error correction is linear, it suffices to show we can correct each Pauli separately, which the syndrome table above does exhaustively.

1. I **(no error):** Syndrome is all $+1$; no correction needed.
2. X_j **(bit-flip on qubit j):** Only the two bit-flip stabilizers of the block containing j fire. Look up the pair in the block table to identify j exactly. Apply X_j.
3. Z_j **(phase-flip on qubit j):** All six bit-flip stabilizers stay at $+1$. Look up (g_7, g_8) in the outer table to identify the block. Apply Z to any qubit in that block.
4. $Y_j = iX_jZ_j$ **(combined error on qubit j):** Both syndromes fire simultaneously. Identify the qubit from the bit-flip syndrome, identify the block from the phase syndrome, apply X_j and Z_j (equivalently Y_j).

Key Point

The syndromes for X, Z, and Y errors on every qubit are all **distinct**, covering every non-trivial syndrome pattern. The Shor code therefore corrects **any single-qubit error on any of the 9 qubits** while preserving the logical qubit perfectly.

4.5 Logical Operators

The operators that act *logically* on the encoded qubit without being detected by the stabilizers are:

$$\bar{X} = X_1X_2X_3X_4X_5X_6X_7X_8X_9 = X^{\otimes 9},$$
$$\bar{Z} = Z_1Z_2Z_3.$$

Intuition

$\bar{X} = X^{\otimes 9}$ commutes with all 8 stabilizer generators and flips the logical bit: $|0\rangle_L \leftrightarrow |1\rangle_L$. $\bar{Z} = Z_1Z_2Z_3$ commutes with all stabilizers and introduces a relative phase between $|0\rangle_L$ and $|1\rangle_L$. Any product of one Z from each block works equally well, e.g. $Z_1Z_4Z_7$ or $Z_2Z_5Z_8$.

Exercise 4.1. Verify that g_7 and g_8 commute. (*Hint:* they share 3 common X-positions; count how many positions they anticommute.)

Exercise 4.2. Show that $\bar{Z} = Z_1Z_2Z_3$ commutes with all 8 stabilizer generators but is not itself in the stabilizer group.

Exercise 4.3. Why does the Shor code use 9 qubits and not 7? Research (or argue from first principles) why the code distance must be at least 3 to correct any single-qubit error.

Section Summary

Section 4 key points:

- The Shor code combines a phase-flip code (3 blocks) with a bit-flip code (within each block) to correct any single-qubit error.
- It uses 9 physical qubits to encode 1 logical qubit.

- 8 stabilizer generators (6 ZZ-type, 2 $XXXXXX$-type) produce 8 syndrome bits, uniquely identifying all single-qubit Pauli errors.
- Any continuous single-qubit error is corrected via discretisation.

5 The Steane $[7, 1, 3]$ Code

Intuition

The Shor code encodes 1 logical qubit using 9 physical qubits. That works, but it feels wasteful. Can we do better? The **Steane code** achieves the same error-correction power with only **7 physical qubits**. To understand how, we first need to look at a classical error-correcting code called the **Hamming code**, which has been used in computer memory chips since the 1950s. The Steane code is essentially the Hamming code "lifted" to the quantum world.

5.1 Step 1 — The Classical Hamming $[7, 4, 3]$ Code

5.1.1 What is a classical code?

Imagine you want to send 4 bits of information over a noisy channel that might flip one of your bits. If you send the 4 bits raw, a single flip gives you wrong information and you cannot even tell where the error occurred.

A **classical error-correcting code** solves this by adding *redundant* bits (called **parity bits**) before sending. The Hamming code adds 3 extra parity bits to your 4 data bits, producing a 7-bit **codeword**. The key promise is:

> *"If at most one of the 7 transmitted bits is flipped, the receiver can figure out* which *bit was flipped and correct it."*

The parameters of the code are written $[n, k, d] = [7, 4, 3]$:

- $n = 7$: total bits in a codeword,
- $k = 4$: data (information) bits,
- $d = 3$: minimum distance — any two valid codewords differ in at least 3 positions, so $\lfloor (3-1)/2 \rfloor = 1$ error can always be corrected.

5.1.2 Two complementary tools: G and H

The Hamming code is described by two matrices that work as a pair.

Matrix	Role	How you use it
Generator matrix G (4×7)	**Encoding**: turns your message into a valid codeword.	Input: 4-bit message $\mathbf{m}$. Compute $\mathbf{c} = \mathbf{m}G \pmod 2$. Output: valid 7-bit codeword $\mathbf{c}$.
Parity-check matrix $H(3 \times 7)$	**Checking/decoding**: detects and locates errors.	Input: received 7-bit word $\mathbf{r}$. Compute syndrome $\mathbf{s} = H\mathbf{r}^T \pmod 2$. If $\mathbf{s} = \mathbf{0}$: no error. Otherwise: $\mathbf{s}$ identifies the flipped bit.

Think of G as the **sender's tool** and H as the **receiver's tool**. They are designed together so that every codeword produced by G is automatically accepted by H:

$$HG^T = 0 \pmod 2.$$

This guarantees that if no error occurs, the syndrome is always zero.

5.1.3 Valid codewords: not every 7-bit string qualifies

Out of $2^7 = 128$ possible 7-bit strings, only $2^4 = 16$ are valid codewords. Those 16 are precisely the strings that pass all three parity checks imposed by H.

The parity-check constraints are *not* automatically satisfied by arbitrary bit strings — they are conditions that the encoder deliberately builds into the codeword using G. This is the foundation of the whole scheme:

- The **sender** uses G to construct one of the 16 valid codewords from the 4-bit message.
- The **receiver** uses H to check whether the received string is still one of those 16. If it is not, at least one bit was flipped in transit.

5.1.4 The generator matrix: constructing a codeword

The generator matrix for this Hamming code is:

$$G = \begin{pmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

The last four columns of G form a 4×4 identity block. This means bits 4–7 of every codeword are exactly your 4 original data bits, and the first three bits are the parity bits that G computes for you.

To encode a message $\mathbf{m} = (m_1, m_2, m_3, m_4)$, XOR together the rows of G wherever the corresponding message bit is 1:

$$\mathbf{c} = \mathbf{m}\,G \pmod 2 = m_1\,(\text{row}_1) \oplus m_2\,(\text{row}_2) \oplus m_3\,(\text{row}_3) \oplus m_4\,(\text{row}_4).$$

Worked Example 5.1: Encoding $\mathbf{m} = (0,0,0,0)$ and $\mathbf{m} = (1,1,1,1)$

Message $\mathbf{m} = (0,0,0,0)$. All message bits are 0, so no rows are selected.

$$\mathbf{c} = (0,0,0,0,0,0,0).$$

The all-zeros message gives the all-zeros codeword.

Message $\mathbf{m} = (1,1,1,1)$. All message bits are 1, so XOR all four rows of G:

$$\begin{aligned} \text{row}_1 &= (1,1,0,1,0,0,0) \\ \text{row}_2 &= (0,1,1,0,1,0,0) \\ \text{row}_3 &= (1,0,1,0,0,1,0) \\ \text{row}_4 &= (1,1,1,0,0,0,1) \\ \hline \mathbf{c} &= (1,1,1,1,1,1,1) \pmod 2. \end{aligned}$$

Notice that bits 4–7 are $(1,1,1,1)$, which are the original data bits, and bits 1–3 are the computed parity bits $(1,1,1)$.

Verify using H:

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

For $\mathbf{c} = (1,1,1,1,1,1,1)$, every entry is 1, so we sum all 7 columns of H:

$$\begin{aligned} \text{Row 1 check:} \quad & 1+0+0+1+0+1+1 = 4 \equiv 0 \pmod 2. \\ \text{Row 2 check:} \quad & 0+1+0+1+1+0+1 = 4 \equiv 0 \pmod 2. \\ \text{Row 3 check:} \quad & 0+0+1+0+1+1+1 = 4 \equiv 0 \pmod 2. \end{aligned}$$

All three checks pass, confirming $(1,1,1,1,1,1,1)$ is a valid codeword. ✓

5.1.5 The parity-check matrix H in detail

$$H = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{pmatrix}.$$

Each **row** specifies one parity check. The 1s in a row tell you which bit positions are included in that check. Row i of $H\mathbf{c}^T$ computes the XOR of the bits at those positions; a valid codeword must give 0 on every row.

- **Row 1** checks bits 1, 4, 6, 7: $c_1 \oplus c_4 \oplus c_6 \oplus c_7 = 0$.
- **Row 2** checks bits 2, 4, 5, 7: $c_2 \oplus c_4 \oplus c_5 \oplus c_7 = 0$.
- **Row 3** checks bits 3, 5, 6, 7: $c_3 \oplus c_5 \oplus c_6 \oplus c_7 = 0$.

These three equations are the constraints that G builds into every codeword it produces. A random 7-bit string will almost certainly violate at least one of them.

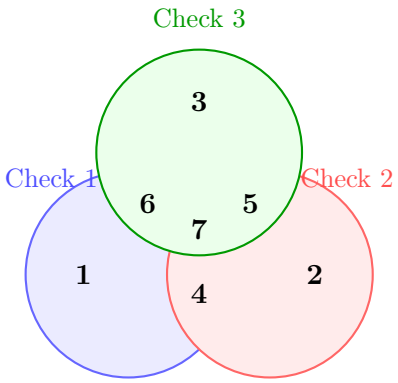


Figure 10: The three parity-check groups of the Hamming code. Each circle shows which bit positions participate in that check. Bit 7 sits in the centre because it is included in all three checks.

5.1.6 The syndrome: pinpointing the error

Suppose a valid codeword $\mathbf{c}$ is sent but a single bit (bit j) is flipped in transit. The receiver gets $\mathbf{r} = \mathbf{c} \oplus \mathbf{e}$, where $\mathbf{e}$ is the all-zero vector except for a 1 in position j. The receiver computes the **syndrome**:

$$\mathbf{s} = H\mathbf{r}^T = H(\mathbf{c} \oplus \mathbf{e})^T = \underbrace{H\mathbf{c}^T}_{=\mathbf{0}} \oplus H\mathbf{e}^T = H\mathbf{e}^T \pmod 2.$$

Because $H\mathbf{c}^T = \mathbf{0}$ for any valid codeword, the syndrome depends *only on the error*, not on which message was sent. This is what makes decoding unambiguous.

Since $\mathbf{e}$ has exactly one 1 (at position j), the product $H\mathbf{e}^T$ picks out exactly the j-th column of H.

The key property: the 7 columns of H are all 7 possible nonzero 3-bit strings, each one different. So every single-bit error produces a *unique* syndrome, and the syndrome tells the receiver exactly which bit was flipped.

Worked Example 5.2: Error detection and correction

Setup. The sender transmits the all-zeros codeword $\mathbf{c} = (0,0,0,0,0,0,0)$ (message $\mathbf{m} = (0,0,0,0)$). During transmission, **bit 4 is flipped**. The receiver gets $\mathbf{r} = (0,0,0,\mathbf{1},0,0,0)$.

Step 1: Compute the syndrome. Only bit 4 of $\mathbf{r}$ is 1, so the syndrome is just column 4 of H:

$$\mathbf{s} = H\mathbf{r}^T = 1 \cdot \underbrace{\begin{pmatrix}1\\1\\0\end{pmatrix}}_{\text{column 4 of } H} = \begin{pmatrix}1\\1\\0\end{pmatrix}.$$

Step 2: Identify the flipped bit. We look up which column of H equals the syndrome $(1,1,0)^T$:

Bit position j	1	2	3	4	5	6	7
Column j of H	$\begin{pmatrix}1\\0\\0\end{pmatrix}$	$\begin{pmatrix}0\\1\\0\end{pmatrix}$	$\begin{pmatrix}0\\0\\1\end{pmatrix}$	$\begin{pmatrix}1\\1\\0\end{pmatrix}$	$\begin{pmatrix}0\\1\\1\end{pmatrix}$	$\begin{pmatrix}1\\0\\1\end{pmatrix}$	$\begin{pmatrix}1\\1\\1\end{pmatrix}$

The syndrome $(1,1,0)^T$ matches **column 4**. The error is on **bit 4**.

Step 3: Correct the error. Flip bit 4 of $\mathbf{r}$ back to 0, recovering $\mathbf{c} = (0,0,0,0,0,0,0)$ and therefore message $\mathbf{m} = (0,0,0,0)$.

Why does this always work? The 7 columns of H are all 7 nonzero 3-bit strings, each distinct. Every single-bit error therefore produces a unique syndrome, and the syndrome unambiguously identifies the flipped bit.

5.2 Step 2 — The CSS Construction: Going Quantum

5.2.1 Why can't we just use the classical code directly?

In quantum computing, qubits can suffer two independent kinds of single-qubit error:

- X **errors** (bit-flip): $|0\rangle \leftrightarrow |1\rangle$. Analogous to classical bit-flips.
- Z **errors** (phase-flip): $|+\rangle \leftrightarrow |-\rangle$. No classical analogue.

We need a code that detects *both* types simultaneously without collapsing the encoded quantum state.

5.2.2 The CSS idea: two codes in one

Calderbank, Shor, and Steane discovered that if a classical code satisfies a certain self-compatibility condition, it can be doubled up to protect against both kinds of quantum errors at once.

Definition 5.1 (CSS code)**.** Given classical linear codes $C_1 \supseteq C_2$ with parity-check matrices H_1 and H_2, the **CSS code** $\mathrm{CSS}(C_1, C_2)$ has stabiliser generators:

X-type: one generator per row of H_2, placing X on each qubit where the row has a 1,

Z-type: one generator per row of H_1, placing Z on each qubit where the row has a 1.

All generators commute if and only if $H_1 H_2^T = 0 \pmod 2$.

Why does $H_1 H_2^T = 0$ guarantee commutativity? An X-generator and a Z-generator commute if and only if they share an *even* number of qubits (two anticommutations on the same qubit cancel each other, restoring a $+1$ sign). The (i, j)-entry of $H_1 H_2^T$ counts (mod 2) how many qubits are shared by the i-th Z-generator and the j-th X-generator. Setting this to 0 forces every such pair to overlap evenly, and hence to commute.

5.3 Step 3 — Building the Steane Code

The Steane code uses $H_1 = H_2 = H$ (the same Hamming parity-check matrix for both X- and Z-type generators).

5.3.1 Checking the CSS condition

We need $HH^T = 0 \pmod 2$. As a spot-check, row 1 dotted with row 2:

$$(1,0,0,1,0,1,1) \cdot (0,1,0,1,1,0,1)^T = 0+0+0+1+0+0+1 = 2 \equiv 0 \pmod 2.$$

All nine entries work out to 0 (mod 2) (a special property of the Hamming code called **self-orthogonality**), so the CSS condition is satisfied.

5.3.2 The six stabiliser generators

Each of the 3 rows of H gives one X-generator and one Z-generator, for 6 generators in total:

Row	Positions of 1s	X-generator	Z-generator
1	1, 4, 6, 7	$g_1 = X_1X_4X_6X_7$	$g_4 = Z_1Z_4Z_6Z_7$
2	2, 4, 5, 7	$g_2 = X_2X_4X_5X_7$	$g_5 = Z_2Z_4Z_5Z_7$
3	3, 5, 6, 7	$g_3 = X_3X_5X_6X_7$	$g_6 = Z_3Z_5Z_6Z_7$

These 6 stabilisers reduce the 7-qubit Hilbert space from $2^7 = 128$ dimensions to $2^{7-6} = 2$ dimensions — just enough room for one logical qubit.

5.3.3 Commutation, anticommutation, and why they matter

Before explaining how errors are detected, we need to understand one crucial point: what it means for two operators to **commute** or **anticommute**, and why that determines the measurement outcome.

The basic Pauli anticommutation relation. The single-qubit Pauli operators X and Z satisfy:

$$XZ = -ZX.$$

Applying X then Z gives the *opposite sign* to applying Z then X. We say X and Z **anticommute**. By contrast, $ZZ = ZZ$ and $XX = XX$, so same-type pairs **commute**.

Stabilisers and measurement outcomes. The encoded state $|\psi\rangle$ is defined to be a $+1$ eigenstate of every stabiliser generator g:

$$g\,|\psi\rangle = +1 \cdot |\psi\rangle.$$

Measuring g on the error-free state always gives $+1$ — the "all clear" signal.

How an error changes the measurement outcome. Suppose error E has occurred, so the state is now $E|\psi\rangle$. Measuring g on this state gives:

Relationship between E and g	**Outcome**	**Why**
$gE = Eg$ (commute)	$+1$	$g(E\|\psi\rangle) = E(g\|\psi\rangle) = E(+1\|\psi\rangle) = +1 \cdot (E\|\psi\rangle)$
$gE = -Eg$ (anticommute)	-1	$g(E\|\psi\rangle) = -E(g\|\psi\rangle) = -E(+1\|\psi\rangle) = -1 \cdot (E\|\psi\rangle)$

A -1 outcome is the flag that an error has occurred. Crucially, this measurement does *not* collapse or disturb the encoded logical information — it only reveals which error happened.

The rule for multi-qubit stabilisers. On a single qubit, X and Z anticommute; operators on *different* qubits always commute. Therefore, for a multi-qubit error E and stabiliser g:

Situation	**Outcome of measuring g**
E and g share no qubit	$+1$ (commute)
E and g share qubits, but same Pauli type (X with X, or Z with Z)	$+1$ (commute)
E and g share an *odd* number of qubits where their Pauli types differ (X vs Z)	-1 (anticommute)

The punchline: a stabiliser flips to -1 precisely when it "sees" the error on an odd number of qubits. By recording which generators give -1, we obtain a binary syndrome — exactly like the Hamming syndrome, but now operating on quantum operators.

5.3.4 How errors are detected

Detecting a Z error (phase-flip). A Z error on qubit j behaves as follows against each generator:

- An X-generator that *includes* qubit j has an X on qubit j. Since $ZX = -XZ$, they anticommute there $\Rightarrow$ outcome -1.
- An X-generator that *excludes* qubit j has no X on qubit j, so it commutes with Z_j $\Rightarrow$ outcome $+1$.
- All Z-generators commute with Z_j (same type everywhere) $\Rightarrow$ outcome $+1$ always.

The pattern of -1 outcomes among g_1, g_2, g_3 is precisely the Hamming syndrome for an error on bit j. The Hamming decoder identifies j.

Detecting an X error (bit-flip). By symmetry (swap X and Z throughout): an X error on qubit j anticommutes with every Z-generator that includes qubit j, and commutes with everything else. The Z-syndrome identifies j.

Worked Example 5.3: Detecting a Z-error on qubit 3

The error. A phase-flip Z_3 occurs on qubit 3.

Measure the X-type generators. Z_3 anticommutes with X only on qubit 3.

$$\begin{aligned} g_1 = X_1X_4X_6X_7: &\quad 3 \notin \{1,4,6,7\} \Rightarrow \text{commutes} \Rightarrow \text{outcome } +1. \\ g_2 = X_2X_4X_5X_7: &\quad 3 \notin \{2,4,5,7\} \Rightarrow \text{commutes} \Rightarrow \text{outcome } +1. \\ g_3 = X_3X_5X_6X_7: &\quad 3 \in \{3,5,6,7\} \Rightarrow \text{anticommutes} \Rightarrow \text{outcome } -1. \end{aligned}$$

Measure the Z-type generators. Z_3 commutes with all Z-generators (same Pauli type), so all give $+1$.

Decode the syndrome. Map $+1 \to 0$ and $-1 \to 1$:

$$(g_1, g_2, g_3) = (+1, +1, -1) \longrightarrow (0, 0, 1).$$

Column 3 of H is $(0,0,1)^T$, which matches exactly. The error is on **qubit 3**.

Correction. Apply Z_3. The encoded state is restored without ever measuring the logical qubit directly.

Key Point

The Steane code runs the Hamming syndrome decoder *twice in parallel* — once to find Z errors (using X-generator outcomes) and once to find X errors (using Z-generator outcomes). The two syndromes are completely independent and do not interfere with each other.

5.4 Code Parameters and Logical Operators

5.4.1 Parameters

Parameter	Value	What it means
n	7	Total physical qubits used.
k	1	Logical qubits encoded. $k = n -$ (no. of generators) $= 7 - 6 = 1$.
d	3	Code distance. We can correct $\lfloor (d-1)/2 \rfloor = 1$ arbitrary single-qubit error.

5.4.2 Logical operators

Every quantum code must have **logical operators** that act on the encoded qubit without being detected by any stabiliser. For the Steane code:

$$\bar{X} = X^{\otimes 7}, \qquad \bar{Z} = Z^{\otimes 7}.$$

Both act on all 7 qubits, consistent with $d = 3$: an adversary must corrupt at least 3 qubits simultaneously to implement an undetectable logical operation.

5.5 Transversal Gates: A Major Advantage

A logical gate is **transversal** if it is implemented by applying the same operation independently to each physical qubit. Transversal gates cannot spread errors between qubits during the gate, making them inherently fault-tolerant.

Logical gate	Physical implementation	Why it works
$\bar{H}$ (Hadamard)	$H^{\otimes 7}$	The Steane code is self-dual ($H_X = H_Z = H$), so H on every qubit swaps X-generators with Z-generators, mapping the code back to itself.
$\bar{S}$ (Phase, $S = \mathrm{diag}(1, i)$)	$S^{\otimes 7}$	S maps $X \to Y = iXZ$ and $Z \to Z$, preserving the CSS stabiliser structure.
$\overline{\mathrm{CNOT}}$	$\mathrm{CNOT}^{\otimes 7}$	Bitwise CNOT between two 7-qubit Steane code blocks.

Together, $\bar{H}$, $\bar{S}$, $\overline{\mathrm{CNOT}}$ generate the **Clifford group** — powerful but not universal.

Caution

The T gate ($T = \mathrm{diag}(1, e^{i\pi/4})$) is *not* transversal in the Steane code: $T^{\otimes 7}$ does not implement a logical $\bar{T}$. A fault-tolerant $\bar{T}$ requires **magic state distillation** (Section 8.4), the single largest overhead cost in fault-tolerant quantum computing.

5.6 Summary: Shor vs. Steane

	Shor code	Steane code
Physical qubits n	9	7
Logical qubits k	1	1
Code distance d	3	3
Errors correctable	1	1
Transversal Clifford gates?	Partial	Yes (full Clifford group)
Key idea	Concatenation	CSS + Hamming code

Both codes correct any single-qubit error, but the Steane code does so with fewer qubits and a richer set of transversal gates, making it the more practical choice and a foundational building block for modern fault-tolerant architectures.

Exercise 5.1. Verify that $HH^T = 0 \pmod 2$ for the Hamming matrix H given above. This is the CSS construction validity condition.

Exercise 5.2. Compute the Z-syndrome (outcomes of g_4, g_5, g_6) when an X-error occurs on qubit 5. Confirm it correctly identifies qubit 5.

Exercise 5.3. Explain why the Steane code needs only 7 qubits while the Shor code needs 9 to correct the same set of errors. What property of the Hamming code makes this possible?

Section Summary

Section 5 key points:

- The Steane code is a $[\![7, 1, 3]\!]$ CSS code derived from the classical Hamming $[7, 4, 3]$ code.

- Its 6 stabilizer generators (3 X-type, 3 Z-type) come directly from the rows of the Hamming parity-check matrix.
- X-type and Z-type generators always commute in a CSS code.
- The code admits transversal Hadamard, Phase, and CNOT — the full Clifford group — but not the T gate.

6 The Stabilizer Formalism

Intuition

The 3-qubit, Shor, and Steane codes all share a common structure: there is a set of multi-qubit Pauli operators (the *stabilizers*) that all codewords satisfy simultaneously. Errors shift codewords out of this "eigenspace", and measuring the stabilizers — without touching the logical information — tells you what went wrong. The stabilizer formalism is the mathematical framework that unifies all these examples and provides a systematic way to construct and analyse quantum codes.

6.1 The Pauli Group (Reminder)

The n-qubit Pauli group $\mathcal{P}_n$ was defined in Section 2. Its key properties:

- Every element squares to $\pm I$.
- Any two elements either commute or anticommute.
- Elements can be described efficiently: an n-qubit Pauli is specified by two n-bit strings (a, b) where $a_j = 1$ means X_j appears and $b_j = 1$ means Z_j appears (with $Y_j = iX_jZ_j$ when both are 1), plus a phase.

6.2 Definition of a Stabilizer Code

Definition 6.1 (Stabilizer group and code space)**.** An abelian subgroup $S \leq \mathcal{P}_n$ not containing $-I$ is called a **stabilizer group**. The associated **code space** is the simultaneous $+1$ eigenspace:

$$\mathcal{C}(S) = \left\{ |\psi\rangle \in (\mathbb{C}^2)^{\otimes n} \;\middle|\; s\,|\psi\rangle = +\,|\psi\rangle\,,\ \forall s \in S \right\}.$$

If S is generated by $n-k$ independent generators $\{g_1, \ldots, g_{n-k}\}$, then $\dim \mathcal{C}(S) = 2^k$, encoding k logical qubits into n physical qubits.

Worked Example 6.1: Verifying the 3-qubit code space

The 3-qubit bit-flip code has stabilizer $S = \langle Z_1Z_2, Z_2Z_3 \rangle$. We check that $|000\rangle$ and $|111\rangle$ are in the code space:

$$\begin{aligned} Z_1Z_2\,|000\rangle &= (+1)(+1)\,|000\rangle = +\,|000\rangle\,, \\ Z_2Z_3\,|000\rangle &= (+1)(+1)\,|000\rangle = +\,|000\rangle\,, \\ Z_1Z_2\,|111\rangle &= (-1)(-1)\,|111\rangle = +\,|111\rangle\,, \\ Z_2Z_3\,|111\rangle &= (-1)(-1)\,|111\rangle = +\,|111\rangle\,. \end{aligned}$$

Any superposition $\alpha\,|000\rangle + \beta\,|111\rangle$ is therefore also in the code space. We have $n = 3$, 2 generators, so $k = 3 - 2 = 1$ logical qubit encoded. Correct!

6.3 Logical Operators

Definition 6.2 (Normaliser and logical operators)**.** The **normaliser** of S in $\mathcal{P}_n$ is:

$$N(S) = \{P \in \mathcal{P}_n \mid PSP^{-1} = S\} = \{P \in \mathcal{P}_n \mid [P, s] = 0,\ \forall s \in S\}.$$

Logical operators are elements of $N(S) \setminus S$. They preserve the code space ($P\mathcal{C} = \mathcal{C}$) but act non-trivially on the encoded qubits.

- Logical $\bar{X}_i$ and $\bar{Z}_i$ for the i-th logical qubit satisfy the usual qubit algebra on $\mathcal{C}$.
- An element of $N(S) \setminus S$ is an **undetectable error**: it passes the syndrome check but changes the logical information. This is why it matters that the *code distance* $d = \min_{P \in N(S)\setminus S} \mathrm{wt}(P)$ is large.

6.4 Syndrome Measurement in the Stabilizer Formalism

Setup. For each generator g_i, prepare an ancilla qubit in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, apply a controlled-$g_i$ (with ancilla as control), then measure the ancilla in the X basis.

The measurement outcome $s_i \in \{+1, -1\}$ (equivalently $s_i \in \{0, 1\}$) satisfies:

$$s_i = \begin{cases} +1 & \text{if the error } E \text{ commutes with } g_i \\ -1 & \text{if the error } E \text{ anticommutes with } g_i \end{cases}$$

The full collection $(s_1, \ldots, s_{n-k})$ is the **error syndrome**.

Worked Example 6.2: Complete syndrome calculation: Steane code, X_4 error

Consider the Steane code with generators as in Section 5. Suppose an X-error occurs on qubit 4.
We check each Z-type generator (they detect X-errors):

$$\begin{aligned} g_4 = Z_1Z_4Z_6Z_7: &\quad Z_4 \text{ anticommutes with } X_4 \Rightarrow s_4 = -1. \\ g_5 = Z_2Z_4Z_5Z_7: &\quad Z_4 \text{ anticommutes with } X_4 \Rightarrow s_5 = -1. \\ g_6 = Z_3Z_5Z_6Z_7: &\quad Z_3, Z_5, Z_6, Z_7 \text{ all commute with } X_4 \Rightarrow s_6 = +1. \end{aligned}$$

All X-type generators commute with X_4 (same Pauli type): $s_1 = s_2 = s_3 = +1$.
Syndrome:

$$(s_1, s_2, s_3, s_4, s_5, s_6) = (+1, +1, +1, -1, -1, +1).$$

The Z-syndrome in binary: $(s_4 - 1)/(-2), (s_5 - 1)/(-2), (s_6 - 1)/(-2) = (1, 1, 0) \to$ row pattern matching column 4 of H. This correctly identifies qubit 4. Apply X_4 to correct.

6.5 The Knill–Laflamme Error Correction Conditions

Theorem 6.1 (Knill–Laflamme conditions)**.** *A code with code projector $\Pi = \sum_i |c_i\rangle\langle c_i|$ (sum over an orthonormal basis $\{|c_i\rangle\}$ for $\mathcal{C}$) can correct a set of errors $\{E_a\}$ if and only if:*

$$\langle c_i | E_a^\dagger E_b | c_j \rangle = \delta_{ij} \lambda_{ab}$$

for all i, j and all errors E_a, E_b in the correctable set. Equivalently: $\Pi E_a^\dagger E_b \Pi = \lambda_{ab} \Pi$.

- **Non-degenerate condition ($i \neq j$):** Different codewords lead to orthogonal error spaces. The syndrome measurement can distinguish them.
- **Degenerate condition ($i = j$):** The "size" of each error looks the same from inside the code space. This means no encoded information is leaked.
- **For stabilizer codes:** The conditions reduce to requiring that $E_a^\dagger E_b$ either lies in S (trivial, correctable) or has non-trivial syndrome (detectable).

Worked Example 6.3: KL conditions for the 3-qubit bit-flip code

The code space basis is $\{|0\rangle_L, |1\rangle_L\} = \{|000\rangle, |111\rangle\}$. Consider errors $\{I, X_1, X_2, X_3\}$. Check the KL condition for $E_a = X_1$, $E_b = X_2$:

$$E_a^\dagger E_b = X_1 X_2.$$

$$\begin{aligned}
\langle 000| X_1X_2 |000\rangle &= \langle 000|110\rangle = 0, \\
\langle 111| X_1X_2 |111\rangle &= \langle 111|001\rangle = 0, \\
\langle 000| X_1X_2 |111\rangle &= \langle 000|001\rangle = 0, \\
\langle 111| X_1X_2 |000\rangle &= \langle 111|110\rangle = 0.
\end{aligned}$$

So $\Pi X_1 X_2 \Pi = 0$, which satisfies the KL condition with $\lambda_{12} = 0$. (The two errors lead to orthogonal error spaces.)

6.6 Graphical Summary: Stabilizer Code Structure

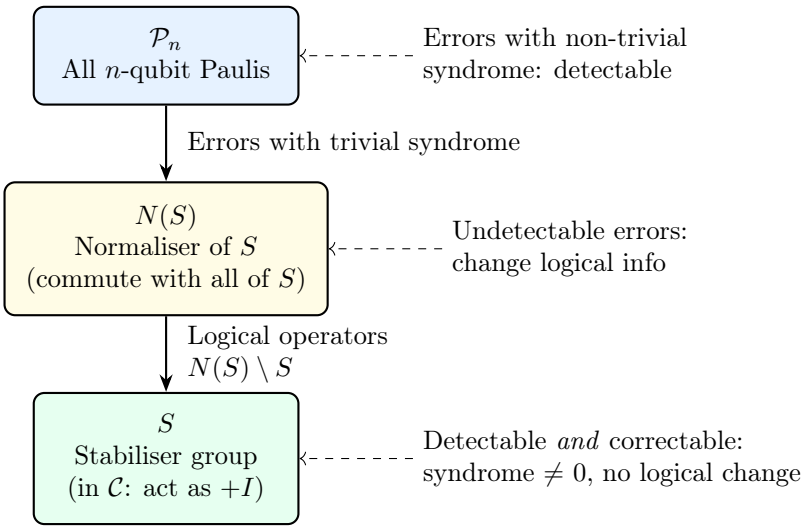


Figure 11: Hierarchy of the Pauli group relative to the stabiliser S. Errors in $\mathcal{P}_n \setminus N(S)$ are detectable; errors in $N(S) \setminus S$ are logical operators (undetectable and harmful); errors in S act trivially.

Exercise 6.1. For the 3-qubit bit-flip code, show that $X_1X_2X_3 \in N(S) \setminus S$ and that it acts as the logical $\bar{X}$ operator.

Exercise 6.2. Verify the KL conditions for the 3-qubit bit-flip code for the pair $E_a = X_1$, $E_b = X_1$ (the "diagonal" case $a = b$).

Exercise 6.3. Show that the Steane code satisfies the KL conditions for single-qubit Z-errors by checking that $\Pi Z_i^\dagger Z_j \Pi = 0$ for $i \neq j$ and $\Pi Z_i^2 \Pi = \Pi$ for any qubit i.

Section Summary

Section 6 key points:

- A stabilizer code is defined by an abelian Pauli subgroup S; the code space is the $+1$ eigenspace of all generators.

- n physical qubits, $n-k$ generators $\Rightarrow$ k logical qubits.
- Syndrome measurement projects errors onto Pauli components without disturbing logical information.
- KL conditions: a code corrects error set $\{E_a\}$ iff projected error products are proportional to the code projector.
- Undetectable errors $(N(S) \setminus S)$ act as logical operators; their minimum weight is the code distance d.

7 The Surface Code

Intuition

The surface code is the most promising candidate for practical quantum computers. Imagine a checkerboard. Data qubits sit on the *edges* of the board (the lines between squares). Two kinds of stabilizers sit on the squares themselves: "plaquette" stabilizers (on dark squares) measure Z parities and detect X errors; "vertex" stabilizers (on light squares) measure X parities and detect Z errors. An error creates a pair of "defects" on adjacent squares — like flipping two squares on the checkerboard — and correcting the error means pairing up defects and erasing them.

7.1 Lattice Structure

Consider a $d \times d$ square lattice where d is the **code distance**:

- **Data qubits** sit on the edges of the lattice (horizontal and vertical lines between vertices).
- **Z-stabilizers (plaquettes):** Each interior face of the lattice has a 4-qubit Z stabilizer $Z_{e_1} Z_{e_2} Z_{e_3} Z_{e_4}$, where $e_1, \ldots, e_4$ are the four edges bounding that face.
- **X-stabilizers (vertices):** Each interior vertex has a 4-qubit X stabilizer $X_{e_1} X_{e_2} X_{e_3} X_{e_4}$, where the edges are the four incident edges.
- Boundary qubits participate in 2- or 3-qubit stabilizers.

7.1.1 Counting for a $d = 3$ Surface Code

For $d = 3$, the lattice has $d^2 + (d-1)^2 = 9 + 4 = 13$ data qubits.

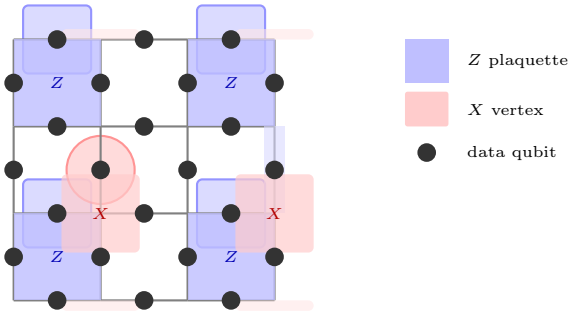


Figure 12: A $d = 3$ surface code lattice (schematic). Black dots are data qubits on edges. Blue squares are Z-stabilizer plaquettes; red regions are X-stabilizer vertices. This encodes 1 logical qubit in 13 data qubits with distance 3.

7.1.2 Code Parameters

For distance d:

$$\text{Data qubits: } n = d^2 + (d-1)^2 = 2d^2 - 2d + 1 \tag{7}$$

$$\text{Z-stabilizers: } (d-1)^2 + (d-1) = d(d-1) \tag{8}$$

$$\text{X-stabilizers: } d(d-1) \tag{9}$$

$$\text{Logical qubits: } k = 1 \quad \text{(for planar surface code)} \tag{10}$$

$$\text{Code distance: } d \quad \text{(minimum weight logical operator)} \tag{11}$$

Worked Example 7.1: Parameters for $d = 5$

$$n = 2(25) - 10 + 1 = 41 \text{ data qubits.}$$

Number of Z-stabilizers: $5 \times 4 = 20$.
Number of X-stabilizers: $5 \times 4 = 20$.
Total stabilizer measurements: $20 + 20 = 40$.
Logical qubits: $41 - 40 = 1$.
Code distance: $d = 5$, so we can correct up to $\lfloor (5-1)/2 \rfloor = 2$ errors.

7.2 Error Correction: Minimum-Weight Perfect Matching

How errors create syndromes. A chain of X-errors on a sequence of data qubits creates *defects* (syndrome -1 outcomes) at the *endpoints* of the chain.

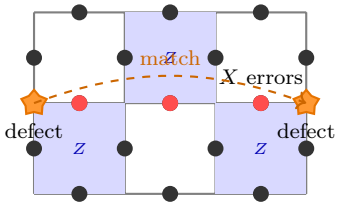


Figure 13: A chain of X-errors (red dots) on three data qubits creates *defects* (orange stars) only at the endpoints. The MWPM algorithm pairs up defects and infers the most likely error chain.

Minimum-weight perfect matching (MWPM).

1. Measure all stabilizers; collect the positions of defects (-1 outcomes).
2. Defects come in pairs (by the topology of the lattice).
3. Find the **minimum-weight perfect matching** — pair up defects so that the total number of data qubits inferred to have errors is minimised (using the Blossom algorithm or similar).
4. Apply the inferred correction.

7.3 Fault-Tolerance Threshold

Theorem 7.1 (Surface code threshold)**.** *For the surface code under depolarising noise with error rate p per gate:*

$$p_L \propto \left(\frac{p}{p_{\text{th}}} \right)^{\lfloor (d+1)/2 \rfloor},$$

*where $p_{\text{th}} \approx 1\%$ for standard depolarising noise. The logical error rate decreases **exponentially** with code distance d provided $p < p_{\text{th}}$.*

Worked Example 7.2: Logical error rate for $d = 5$

Take $p = 0.005$ (0.5%) and $p_{\text{th}} = 0.01$ (1%). Then $p/p_{\text{th}} = 0.5$ and:

$$p_L \propto (0.5)^{\lfloor 6/2 \rfloor} = (0.5)^3 = 0.125.$$

For $d = 7$: $p_L \propto (0.5)^4 = 0.0625$ — a factor 2 improvement per distance step. For $d = 11$:

$(0.5)^6 \approx 0.016$. This shows why large distance (and thus many physical qubits) are needed to reach extremely low logical error rates.

7.4 Logical Gates via Lattice Surgery

Direct transversal gates on the surface code are limited. Logical operations are instead performed by **lattice surgery**:

Logical CNOT: Merge two code patches (measuring joint stabilizers along the interface), perform joint measurements, then split.

Logical Hadamard: On the *rotated* surface code (a 45°-rotated lattice), $H^{\otimes n}$ implements $\bar{H}$.

Logical T gate: Requires magic state distillation (Section 8.4).

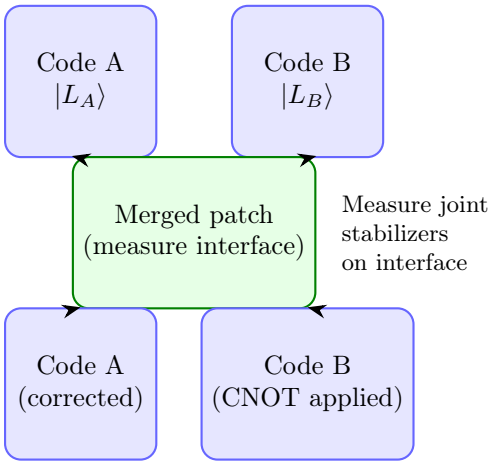


Figure 14: Lattice surgery for a logical CNOT. Two code patches are temporarily merged by measuring joint stabilizers on the interface, then split again. The net effect is a logical CNOT between qubits A and B.

7.5 Why the Surface Code is Practical

- **Local interactions only:** Each stabilizer involves at most 4 nearest-neighbour qubits — perfectly suited to 2D chip architectures (superconducting qubits, trapped ions arranged in a plane).
- **High threshold:** $p_{\text{th}} \approx 1\%$ is achievable with current hardware ($p \sim 0.1$–0.5%).
- **Simple decoding:** MWPM is efficient (polynomial time).
- **Natural fault-tolerance:** Stabilizer measurement errors can be tracked across repeated measurement rounds.

Exercise 7.1. Calculate the number of data qubits, Z-stabilizers, X-stabilizers, and logical qubits for a $d = 7$ surface code. Verify $k = 1$.

Exercise 7.2. A chain of two X-errors occurs on adjacent data qubits in a $d = 3$ surface code. Draw the defect pattern and explain how MWPM would (correctly) decode this error. Now suppose three consecutive X-errors form a chain spanning the lattice. Why does this cause a logical error?

Exercise 7.3. Below the threshold, the logical error rate scales as $p_L \propto (p/p_{\text{th}})^{(d+1)/2}$. How large must d be to achieve $p_L < 10^{-10}$ if $p = 0.003$ and $p_{\text{th}} = 0.01$?

Section Summary

Section 7 key points:

- The surface code places data qubits on lattice edges; Z (plaquette) and X (vertex) stabilizers detect bit- and phase-flip errors.
- A $d = d$ surface code uses $2d^2 - 2d + 1$ qubits, has distance d, and encodes 1 logical qubit.
- Errors create defect pairs; MWPM corrects by pairing defects optimally.
- Threshold $p_{\text{th}} \approx 1\%$; logical error rate drops exponentially with d below threshold.
- Logical gates via lattice surgery; T gate still requires magic state distillation.

8 Fault-Tolerant Quantum Gates

Intuition

Encoding qubits protects stored information, but we also need to *compute* on it. A gate performed directly on physical qubits might spread a single error into many errors — too many for the code to correct. Fault-tolerant gate design is the art of performing logical operations in a way that guarantees a single physical fault produces at most one logical fault.

8.1 Why Naive Gate Application Fails

Consider applying a transversal CNOT to the 3-qubit bit-flip code. A single fault in the gate could spread to affect multiple qubits. Suppose the second CNOT gate has an X error on its control:

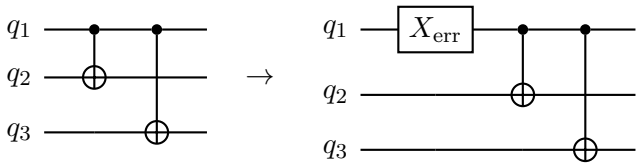


Figure 15: A single error on the first gate in the encoding circuit propagates to affect all subsequent qubits. This "error spreading" is the key danger that fault-tolerant design must prevent.

8.2 Transversal Gates

Definition 8.1 (Transversal gate)**.** A gate U_L is **transversal** if it can be written as $U_L = u^{\otimes n}$ (same single-qubit operation on every physical qubit, possibly with qubit permutations between code blocks).

A transversal gate is automatically fault-tolerant: a fault on physical qubit j affects only qubit j in the encoded block — it cannot spread to other qubits.

Code	Transversal gates	Missing gates
3-qubit bit-flip	$\overline{X}$, $\overline{Z}$	Phase, T, Hadamard
Steane $[\![7,1,3]\!]$	$\bar{H}$, $\bar{S}$, $\overline{\text{CNOT}}$ (Clifford group)	T gate
Surface code	$\bar{H}$ (rotated), $\overline{\text{CNOT}}$ via surgery	T gate

Theorem 8.1 (Eastin–Knill theorem)**.** *No quantum error-correcting code can have a transversal implementation of a* ***universal*** *gate set.*

This means every code must use some non-transversal technique for at least one gate. The T gate ($\pi/8$ gate) is the standard choice.

8.3 Fault-Tolerant Syndrome Measurement

Measuring a multi-qubit stabilizer like $Z_1Z_2Z_3Z_4$ requires ancilla qubits. A naive circuit might look like:

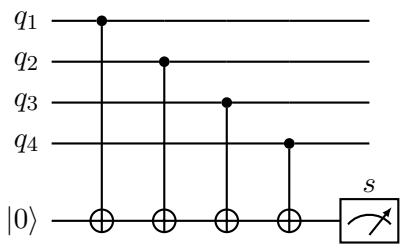


Figure 16: Standard ancilla-based syndrome measurement for the $Z_1Z_2Z_3Z_4$ stabilizer. The ancilla records the parity of the four data qubits. One fault in this circuit can propagate to multiple data qubits.

Caution

A single fault in the ancilla preparation or the CNOT gates can spread to multiple data qubits. For example, an X error on the ancilla before any CNOT will propagate a Z error to *every* data qubit it interacts with (since Z errors on the target propagate to the control). This can defeat the code's error-correcting power. The solution is to use **flag qubits** or **cat states** to detect such spreading.

8.3.1 Repeated Measurement

Even with perfect gates, measurement outcomes themselves are noisy. The standard approach is to repeat stabilizer measurements (typically 3 times for a distance-3 code) and use the majority outcome or track the syndrome history. For the surface code, syndrome measurements are repeated $O(d)$ times to reliably decode.

8.4 Magic State Distillation

8.4.1 Why We Need Magic States

We need a universal gate set. The Clifford gates (which include H, S, CNOT) are transversal in many codes, but they are *not* universal: any circuit built from Clifford gates alone can be simulated classically in polynomial time (Gottesman–Knill theorem).

To achieve universality, we add the T gate:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}.$$

The Clifford group plus T is universal. However, T is not transversal in standard codes.

8.4.2 The Magic State Idea

The T gate can be implemented by **gate teleportation** if we have access to the **magic state**:

$$|T\rangle = T\,|+\rangle = \frac{|0\rangle + e^{i\pi/4}\,|1\rangle}{\sqrt{2}}.$$

The teleportation circuit uses only Clifford gates plus a measurement:

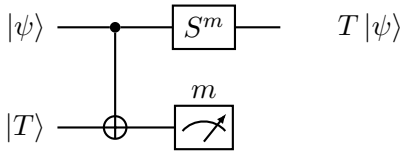


Figure 17: Gate teleportation implementing the T gate. Given one copy of the magic state $|T\rangle$, a Clifford correction S^m (where $m \in \{0, 1\}$ is the measurement result) yields $T|\psi\rangle$ using only Clifford gates.

8.4.3 Distillation Protocol

Physical T gates are noisy; the resulting states are close to (but not exactly) $|T\rangle$. The **15-to-1 distillation protocol** (Bravyi–Kitaev) takes 15 noisy magic states and produces 1 high-fidelity magic state:

- Input: 15 noisy $|T\rangle$ states, each with error probability ϵ.
- Output: 1 magic state with error probability $\approx 35\epsilon^3$ (superquadratic suppression).
- Circuit: uses only Clifford gates (which are fault-tolerant transversally) to distil the high-quality state.

Worked Example 8.1: Distillation improvement

If the physical T gate has error rate $\epsilon = 0.01$:

$$\text{After 1 round: } \epsilon_1 \approx 35(0.01)^3 = 35 \times 10^{-6} = 3.5 \times 10^{-5}.$$
$$\text{After 2 rounds: } \epsilon_2 \approx 35(3.5 \times 10^{-5})^3 \approx 1.5 \times 10^{-12}.$$

Two rounds of 15-to-1 distillation reduce the error from 1% to below 10^{-12} at the cost of $15^2 = 225$ initial magic states.

8.5 The Threshold Theorem

Theorem 8.2 (Fault-Tolerant Threshold Theorem)**.** *Suppose each physical gate, preparation, and measurement has error rate p, and errors are local (independent across qubits). There exists a threshold $p_{\text{th}} > 0$ such that: if $p < p_{\text{th}}$, arbitrarily long quantum computations can be performed with failure probability approaching 0. The qubit overhead scales as poly$(\log(1/\epsilon))$ for target error ϵ.*

- For the surface code: $p_{\text{th}} \approx 1\%$ (depolarising noise).
- For concatenated codes (Section 9): $p_{\text{th}} \approx 10^{-4}$ to 10^{-3}.
- The surface code's higher threshold is a major reason it is the leading practical candidate.

8.5.1 Below vs. Above Threshold

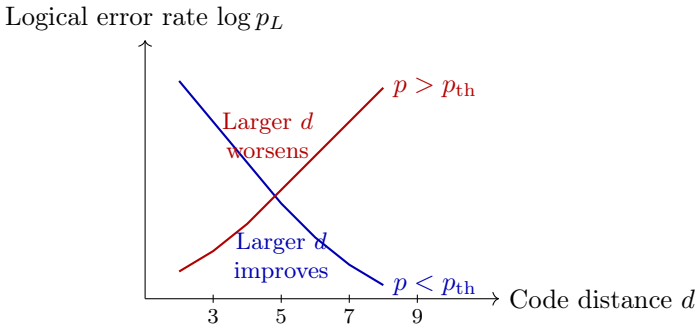


Figure 18: Below the threshold (blue), increasing code distance decreases the logical error rate exponentially. Above the threshold (red), adding more qubits makes things worse because there are too many opportunities for error.

Exercise 8.1. For a $d = 7$ surface code with $p = 0.004$ and $p_{\text{th}} = 0.01$, estimate the logical error rate as a multiple of $(p/p_{\text{th}})^{\lfloor (d+1)/2 \rfloor}$.

Exercise 8.2. Using the gate teleportation circuit of Figure 17, verify that measuring the ancilla in state $|1\rangle$ (outcome $m = 1$) and then applying S to the data qubit produces $T|\psi\rangle$. (*Hint:* work through the action of the CNOT on $|\psi\rangle|T\rangle$, then project onto $m = 0$ and $m = 1$ separately.)

Exercise 8.3. After three levels of 15-to-1 distillation starting from $\epsilon = 0.005$, what is the estimated output magic state error rate? How many physical T-gate preparations does this require?

Section Summary

Section 8 key points:

- Transversal gates prevent error spreading; the Eastin–Knill theorem says no code has a universal transversal gate set.
- Syndrome measurement circuits must be designed carefully to avoid fault propagation; repeated measurement is needed.
- The T gate is implemented via gate teleportation using magic states.
- Magic state distillation (15-to-1 protocol) converts many noisy $|T\rangle$ states into one high-fidelity one.
- The threshold theorem guarantees scalable quantum computation when physical error rates are below p_{th}.

9 Advanced Topics in Quantum Error Correction

Intuition

The surface code is excellent but has one major weakness: it encodes only 1 logical qubit no matter how many physical qubits you use. For a practical quantum computer running a million-gate algorithm, you might need thousands of logical qubits, requiring millions of physical qubits. This section surveys alternative codes that offer better rates (more logical qubits per physical qubit), higher thresholds, or additional transversal gates.

9.1 Quantum Low-Density Parity-Check (LDPC) Codes

Classical LDPC codes are a major tool in classical communications (used in 5G, WiFi, etc.). Each parity-check involves few bits (**sparse**) and each bit appears in few checks — hence "low density".

Quantum LDPC codes generalise this to the stabilizer setting:

- Each stabilizer generator involves at most w qubits (for some constant w, e.g. $w = 6$).
- Each qubit appears in at most w generators.
- Result: sparse syndrome measurement circuits, fewer gates per round.

9.1.1 Recent Breakthrough: Good Quantum LDPC Codes

Code family	**Rate** k/n	**Distance** d	**Stabilizer weight**
Surface code	$\Theta(1/n)$	$\Theta(\sqrt{n})$	4
Hypergraph product	Constant	$\Theta(\sqrt{n})$	$O(1)$
Lifted product (Panteleev–Kalachev 2022)	Constant	$\Theta(n)$	$O(1)$
Fiber bundle codes	Constant	$\Theta(n^{3/5})$	$O(1)$

A code with *constant rate* and *linear distance* ($d = \Theta(n)$) is called a **good quantum LDPC code**. These were only proved to exist in 2022, representing a major theoretical advance.

9.1.2 Practical Challenges

Despite better parameters, quantum LDPC codes face challenges:

- Generators can involve non-local (long-range) qubit pairs — hard to implement in 2D architectures.
- Efficient decoders are still being developed.
- Threshold values and exact overheads are not yet as well understood as for the surface code.

9.2 Color Codes

Color codes are topological codes defined on 3-colorable lattices.

9.2.1 2D Color Code Construction

Consider a 2D lattice where every face can be colored red, green, or blue, and every vertex touches exactly one face of each color. Data qubits sit on the vertices.

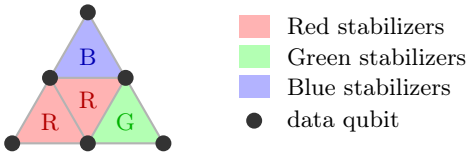


Figure 19: A small portion of a 2D color code lattice. Each colored face contributes both an X-type and a Z-type stabilizer on its vertices. The 3-colorability allows fault-tolerant gates beyond those of the surface code.

For each colored face f, there is an X-stabilizer and a Z-stabilizer on the vertices of f. The code is a $[\![n, 1, d]\!]$ code similar to the surface code.

9.2.2 Advantages of Color Codes

- The 2D color code has the same parameters as the surface code ($d = \Theta(\sqrt{n})$, rate $1/n$) but admits **transversal** H, S, CNOT, *and* T in some 3D variants.
- In 3D color codes, the full Clifford+T gate set is transversal, eliminating the need for magic state distillation — though at the cost of a 3D qubit architecture.
- Syndrome extraction is slightly more complex than for the surface code.

9.3 Concatenated Codes

Definition 9.1 (Code concatenation)**.** Given an "outer" code $C_1 = [\![n_1, k_1, d_1]\!]$ and an "inner" code $C_2 = [\![n_2, k_2, d_2]\!]$ with $k_2 = k_1$, the **concatenated code** $C_1 \circ C_2$ encodes each physical qubit of C_1 using the inner code C_2. The result is a $[\![n_1 n_2, k_1, d_1 d_2]\!]$ code.

Worked Example 9.1: Concatenating two Steane codes

Let both C_1 and C_2 be the Steane $[\![7, 1, 3]\!]$ code. The concatenated code $C_1 \circ C_2$ has:

$$n = 7 \times 7 = 49 \text{ physical qubits},$$
$$k = 1 \text{ logical qubit},$$
$$d = 3 \times 3 = 9.$$

After ℓ levels of concatenation:

$$n_\ell = 7^\ell, \quad d_\ell = 3^\ell, \quad k = 1.$$

Logical error rate under concatenation: Let p be the physical error rate. After ℓ levels:

$$p_L^{(\ell)} \approx \frac{1}{c}(cp)^{2^\ell},$$

where c is a constant depending on the code (for Steane, $c \approx 1/p_{\text{th}}$). For $p = 0.001$ and $c = 100$:

$$\ell = 1: \quad p_L \approx (0.1)^2 = 0.01.$$
$$\ell = 2: \quad p_L \approx (0.1)^4 = 10^{-4}.$$
$$\ell = 3: \quad p_L \approx (0.1)^8 = 10^{-8}.$$

Exponential suppression, but at the cost of $7^3 = 343$ physical qubits per logical qubit at level 3.

9.4 Comparison of QEC Code Families

Code	Rate	Distance	Transversal gates	Architecture
Surface	$\Theta(1/n)$	$\Theta(\sqrt{n})$	Clifford (partial)	2D local
Color (2D)	$\Theta(1/n)$	$\Theta(\sqrt{n})$	Full Clifford	2D local
Color (3D)	$\Theta(1/n)$	$\Theta(n^{1/3})$	Clifford + T	3D local
Quantum LDPC	Constant	up to $\Theta(n)$	Limited	Non-local
Concatenated	Variable	Exponential	Depends	Non-local

Exercise 9.1. After 4 levels of concatenation with a $[\![7, 1, 3]\!]$ code, how many physical qubits encode 1 logical qubit? What is the code distance?

Exercise 9.2. Explain why quantum LDPC codes with constant rate are important for practical quantum computation. What obstacles prevent their immediate use?

Exercise 9.3. A 2D color code and a surface code of the same physical qubit count n both have distance $\Theta(\sqrt{n})$. List two advantages of the color code and one advantage of the surface code.

Section Summary

Section 9 key points:

- Quantum LDPC codes offer constant rate (many logical qubits per physical qubit) with sparse stabilizers; recent breakthroughs achieve linear distance.
- Color codes are topological codes on 3-colorable lattices; 3D variants allow transversal T gates.
- Concatenated codes reduce logical error rates doubly-exponentially in the number of concatenation levels, at exponential qubit cost.
- No single code is optimal for all metrics; the choice depends on hardware architecture, target error rate, and gate set requirements.

10 Summary and Putting It All Together

This section brings together all the major themes of the course.

10.1 The Big Picture

Quantum error correction solves the fundamental problem of quantum computation: qubits decohere and gates are imperfect. The key ideas, in order of development, are:

1. **Errors are continuous, but correctable discretely.** Any single-qubit error decomposes into Pauli operators; syndrome measurement projects errors onto discrete Pauli faults.

2. **Entanglement protects information.** Encoding in entangled states spreads information holographically across many qubits. No individual qubit holds the secret.

3. **Syndromes are the key.** Measuring stabilizers reveals errors without collapsing the logical superposition. This is the central miracle of QEC.

4. **Fault-tolerance requires more than encoding.** Gates, measurements, and state preparations must all be performed in ways that prevent single faults from cascading.

5. **The threshold theorem gives hope.** Provided physical error rates are below $\sim 1\%$, reliable large-scale quantum computation is in principle possible.

10.2 Quick Reference: All Codes Covered

Code	n	k	d	**What it corrects**
3-qubit bit-flip	3	1	1	Single X error
3-qubit phase-flip	3	1	1	Single Z error
Shor 9-qubit	9	1	3	Any single-qubit error
Steane $[\![7, 1, 3]\!]$	7	1	3	Any single-qubit error
Surface code ($d = 5$)	41	1	5	Any 2-qubit error

10.3 Road Map Through the Material

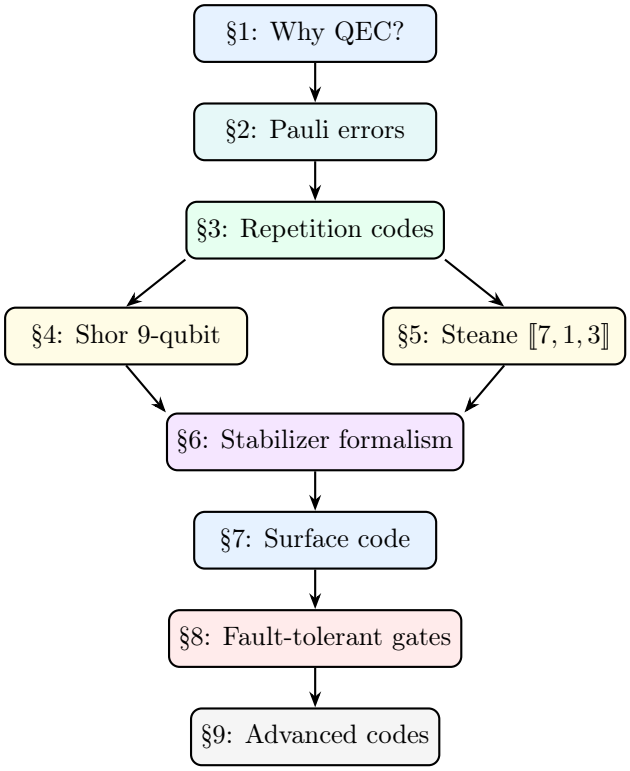


Figure 20: The logical dependency structure of these lecture notes. Each section builds on the previous; Sections 4 and 5 are parallel examples that Section 6 unifies.

10.4 Comprehensive Exercises

These exercises require combining knowledge from multiple sections.

Exercise 10.1. [Full worked pipeline.] A logical qubit is encoded in the 3-qubit bit-flip code. (a) Write the encoded state for $|\psi\rangle = \frac{1}{\sqrt{3}}|0\rangle + \sqrt{\frac{2}{3}}|1\rangle$. (b) Suppose a $Y_2 = iX_2Z_2$ error occurs. Compute the syndrome. (c) Apply the corrective operation and verify the state is restored.

Exercise 10.2. [KL conditions.] Prove that the Shor code satisfies the KL conditions for the error pair $E_a = Z_1$ (phase flip on qubit 1) and $E_b = Z_4$ (phase flip on qubit 4).

Exercise 10.3. [Code comparison.] For a quantum computer needing $p_L = 10^{-12}$ per logical gate with physical error rate $p = 0.001$: (a) How large must d be for the surface code? (b) How many levels of Steane code concatenation achieve this? (c) Which uses fewer physical qubits?

Exercise 10.4. [Steane code X-syndrome.] Suppose Z-errors occur on qubits 2 and 6 simultaneously in the Steane code. Compute the full X-syndrome and determine whether both errors can be identified and corrected. (*Hint:* Check whether the combined syndrome is the XOR of the individual syndromes.)

Exercise 10.5. [Surface code threshold.] A surface code operates at $p = 0.008$ with $p_{\text{th}} = 0.01$. At what code distance d does the logical error rate first drop below 10^{-6}? How many physical qubits does this require?

Exercise 10.6. [Magic state overhead.] A quantum algorithm requires 10^6 T-gate applications. Using the 15-to-1 distillation protocol with two rounds of distillation, and assuming each logical Clifford gate consumes one magic state, how many physical T-gate preparations are needed in total?

Exercise 10.7. [Error channel.] A qubit undergoes a dephasing channel with $p = 0.1$, followed by a bit-flip channel with $q = 0.05$. Write the combined error channel as a sum of Kraus operators and identify the probabilities of each Pauli error.

Exercise 10.8. [Stabilizer design.] Design a simple 5-qubit code (the "perfect code" $[\![5, 1, 3]\!]$) with stabilizers $\{XZZXI, IXZZX, XIXZZ, ZXIXZ\}$ (where we write operators on 5 qubits without tensor product symbols for brevity). (a) Verify that all four generators commute. (b) Show that the code encodes 1 logical qubit. (c) Show that it has distance 3.

Exercise 10.9. [Discretisation revisited.] A qubit suffers a general unitary error $U = \cos\theta\, I + i \sin\theta\, Z$ for $\theta = \pi/6$. The qubit is protected by the 3-qubit phase-flip code. (a) Decompose U in the Pauli basis. (b) After syndrome measurement, what is the probability that no correction is needed? That a correction is applied? (c) Verify that the final state is correct in both cases.

References

[1] Nielsen, M. A. & Chuang, I. L. (2010). *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press. *The standard reference. Chapters 10–11 cover QEC and fault tolerance.*

[2] Shor, P. W. (1995). Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4), R2493. *The original 9-qubit code paper.*

[3] Steane, A. M. (1996). Error correcting codes in quantum theory. *Physical Review Letters*, 77(5), 793. *The* $[\![7, 1, 3]\!]$ *CSS code.*

[4] Knill, E. & Laflamme, R. (1997). Theory of quantum error-correcting codes. *Physical Review A*, 55(2), 900. *The Knill–Laflamme error correction conditions.*

[5] Kitaev, A. Y. (1997). Quantum computations: algorithms and error correction. *Russian Mathematical Surveys*, 52(6), 1191. *The original surface/toric code paper.*

[6] Fowler, A. G., Mariantoni, M., Martinis, J. M., & Cleland, A. N. (2012). Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3), 032324. *Comprehensive surface code reference.*

[7] Bravyi, S. & Kitaev, A. (2005). Universal quantum computation with ideal Clifford gates and noisy ancillas. *Physical Review A*, 71(2), 022316. *The 15-to-1 magic state distillation protocol.*

[8] Panteleev, P. & Kalachev, G. (2022). Asymptotically good quantum and locally testable classical LDPC codes. *Proc. 54th ACM STOC*, 375–388. *Breakthrough result on good quantum LDPC codes.*

[9] Gottesman, D. (2009). *An introduction to quantum error correction and fault-tolerant quantum computation*. arXiv:0904.2557. *Excellent lecture notes covering stabilizer codes and fault tolerance. Freely available.*

Glossary of Key Terms

Ancilla qubit
: A helper qubit used to measure stabilizers without disturbing the data qubits. Prepared in a known state ($|0\rangle$ or $|+\rangle$), coupled to data, then measured.

Code distance d
: The minimum weight (number of non-identity Paulis) of any logical operator. A code of distance d corrects $\lfloor (d-1)/2 \rfloor$ errors.

CSS code
: A code constructed from two classical codes $C_1 \supseteq C_2$ with X-stabilizers from C_2 and Z-stabilizers from C_1. Guarantees X and Z generators commute.

Decoherence
: The loss of quantum coherence (superposition) due to entanglement with the environment. Characterised by times T_1 (energy relaxation) and T_2 (dephasing).

Fault-tolerant gate
: A gate implementation in which a single physical fault creates at most one logical fault.

Logical operator
: A Pauli operator that preserves the code space ($P \in N(S) \setminus S$) but acts non-trivially on the encoded qubit.

Magic state
: A non-stabilizer state $|T\rangle = (|0\rangle + e^{i\pi/4}|1\rangle)/\sqrt{2}$ used via gate teleportation to implement the T gate.

No-cloning theorem
: It is impossible to copy an arbitrary unknown quantum state unitarily. Prevents direct repetition coding of qubits.

Stabilizer
: A Pauli operator g that fixes all codewords: $g\,|\psi\rangle = |\psi\rangle$. Equivalently, a $+1$ eigenstate condition.

Syndrome
: The pattern of ± 1 eigenvalues obtained by measuring all stabilizer generators. Uniquely identifies correctable errors.

Threshold p_{th}
: The maximum physical error rate below which increasing the code distance reduces the logical error rate. Approximately 1% for the surface code.

Transversal gate
: A logical gate implemented by applying the same single-qubit operation independently to each physical qubit. Automatically fault-tolerant.